

PROMPT ENGINEERING FOR NLP TASKS

Handbook

Build a real NLP workflow by directing an AI coding agent

CircleCI Build-Error Intelligence

Built live with Codex • CI/CD Pipeline Failure Logs Case Study

Applied Generative AI and Agentic AI

How to use this guide

This is a hands-on, AI-assisted coding module. You do not hand-write code. You build the entire NLP workflow by prompting an AI coding agent (Codex) from the command line, and then reviewing and refining what it produces. The CircleCI build-error problem is the vehicle; the real skill you are learning is how to direct an AI coding agent well and critically check its work on a real NLP task.

This module sits inside the Applied Generative AI course. It exercises the full practical NLP arc you have seen in the slides: text classification, text-to-text summarisation, evaluation metrics, prompt-engineering fundamentals, and advanced prompting patterns, all against a single dataset, so you see how the pieces connect end to end.

Each of the seven stages gives you six things:

- The NLP concept in plain English.
- The exact prompt you give Codex.
- How to review Codex's output before you accept it.
- A watch-out where the AI tends to make NLP and prompt-design mistakes.
- The numbers you should see when you run it.
- A short reflection to lock in the lesson.

When your numbers differ slightly from the ones printed here, trust your own numbers. The AI generates code fresh each time, so small variation is normal. But the "Watch out" items are non-negotiable correctness checks; do not skip them.

THE ONE IDEA TO CARRY THROUGH THE WHOLE MODULE

Anyone can ask an AI to write code. The professional skill is knowing whether the output it gives back is correct, and, in NLP, whether the prompt you wrote actually produces a useful result. You will see the same task go from useless to useful just by changing the prompt.

1. Before you start: your setup

Set up your environment before you begin so that generated code runs instantly and you spend your time on prompting and reviewing, not on installs.

1. Install the Codex CLI and sign in with your account. Confirm it launches from the terminal and can read and write files in your working directory.
2. Confirm Codex can act on your workspace: open an empty folder, give it a trivial prompt such as "create a file hello.py that prints hello", and confirm it proposes a file edit you can review and accept. This proves the agentic edit-and-review loop works on your machine.
3. Confirm Python 3.10+ is installed (python --version; this guide was built on Python 3.12). Make sure the same interpreter is the one Codex invokes when it runs scripts.
4. Pre-install the packages the workflow needs so generated code runs immediately: pip install pandas openai langchain langchain-openai rouge-score bert-score torch transformers. Then verify the imports succeed.
5. Place the dataset ci_cd_pipeline_failure_logs_dataset.csv in your project folder. This is the one file provided to you. Everything else is generated by Codex as you work.
6. Configure the model gateway. Set the API base and key as environment variables in your shell before launching Codex, and set the model to one your gateway allows. Never commit a key to any file.

WATCH OUT

Keep your API key safe. Set your credentials in the shell environment only. Never paste a real API key into a script, a prompt, or a slide. If the default model is rejected with a "model not allowed" error, set OPENAI_MODEL explicitly to a model your gateway permits. Use a .env file to store your credentials

```
$env:OPENAI_API_KEY="<your_api_key_here>"
$env:OPENAI_API_BASE="<your_gateway_base_url>"
$env:OPENAI_MODEL="<an_allowed_model>"
python -c "import os; print(bool(os.getenv('OPENAI_API_KEY')))"
```

File Edit Selection View Go ... Codex

EXPLORER

CODEX

- .env
- ci_cd_pipeline_failure_logs.csv

ci_cd_pipeline_failure_logs.csv

.env

```
1 OPENAI_API_KEY=[REDACTED]
2 OPENAI_API_BASE=
3 OPENAI_MODEL="gpt-4o-mini"
4
```

CHAT CODEX

Do anything

+ Approve for me 5.2

Ln 4, Col 1 Spaces: 4 UTF-8 CRLF {} Dotenv

2. What you will build: the roadmap

You will build the intelligence layer in seven stages. Each stage produces a script, and every script uses the output of the one before it. Here is the whole journey at a glance.

#	Stage	What you build, and the prompt-engineering lesson
1	Data exploration	Profile the file and confirm the schema. Lesson: trust the data over the spec.
2	PII filtering	Scrub secrets out of every log before any model call. Lesson: constraints protect the business, and redaction quality flows downstream.
3	Classification	Zero-shot vs few-shot on a fixed label set. Lesson: how you ask changes the answer.
4	Summarization	Basic template, then persona + output spec. Lesson: a role and a required shape turn vague text into an actionable fix.
5	Advanced patterns	Chain-of-Thought and self-consistency. Lesson: reasoning and voting buy reliability, but high agreement on no signal is not correctness.
6	Evaluation	Textbook metrics fail honestly; pivot to LLM-as-a-Judge. Lesson: choosing the right metric is itself a skill.
7	Pipeline + feedback loop	Wire it all together, find the weakest summaries, improve the prompt, re-measure. Lesson: prompt engineering is a measure-and-refine loop.

A NOTE ON STAGE 5

Stage 5 (self-consistency) makes five model calls per row, so it is slow and expensive to run repeatedly. Run it once to see the mechanism, then study the saved output rather than re-running it. Every other stage is fast enough to build and re-run freely.

3. Jargon buster: read this first

These are the terms that trip people up most. Come back here whenever one of these words shows up in a stage.

Term	What it actually means (plain English)
Text classification	Sorting a piece of text into one of a fixed set of categories. Here: putting each build failure into a category like dependency failure or test failure.
Zero-shot prompting	Asking the model to do a task with only an instruction and no examples. Fast to write, but often inconsistent.
Few-shot prompting	Adding a handful of worked examples before the real question so the model copies the pattern. Usually more accurate and consistent than zero-shot.
Abstractive summarisation	Generating a new, plain-English explanation of a text rather than copying sentences out of it. This is the text-to-text task in Stage 4.
Persona prompting	Telling the model who to be ("you are a senior DevOps engineer") so its tone and depth match the task.
Output specification	Forcing the answer into a fixed shape (e.g. WHAT FAILED / ROOT CAUSE / SUGGESTED FIX) so it is consistent and parseable.
Chain-of-Thought (CoT)	Asking the model to reason step by step before giving a final answer. Helps on ambiguous inputs where the surface text is misleading.
Self-consistency	Running the same Chain-of-Thought prompt several times and taking a majority vote. Reduces the chance of one unlucky run picking the wrong answer.
ROUGE	A metric that measures word overlap between a generated text and a reference text. Designed for summarisation against a human-written summary.
BERTScore	A metric that measures meaning overlap using embeddings, not just exact words. More forgiving than ROUGE for paraphrased text.
LLM-as-a-Judge	Using a language model to score another model's output against a rubric (e.g. accuracy and actionability from 1 to 5). Useful when there is no reference text.
PII	Personally Identifiable Information: secrets like API keys, tokens, file paths, and credentials that can hide inside logs and must be removed before any external API call.
Hallucination	When the model confidently states something wrong or invented. The reason you always review.
Prompt template	A reusable prompt with blanks (variables) that get filled in per row: the LangChain PromptTemplate used in Stage 4.
Taxonomy / label set	The fixed list of categories the classifier is allowed to choose from.

AI-assisted coding terms

Because you build everything by prompting Codex, you also need these.

Term	What it actually means (plain English)
Codex CLI	An AI coding agent run from the command line. You describe a task in plain language; it reads your project files, writes or edits code, and can run commands. You review its proposed changes before accepting them.
AI coding agent / agentic coding	An AI that does not just autocomplete a line. It can take a whole task ("build a classifier function"), create and edit multiple files, and run code, like a junior developer you direct.
Prompt	The instruction you give the AI. The quality of what you get back depends almost entirely on the quality of the prompt: being specific, giving context, and stating constraints.
Context carry-over	Codex remembers earlier turns in the same session, so you state the business problem once at the start and refer back to it instead of repeating it every stage.
Diff / reviewing a diff	The set of changes the AI proposes, shown as additions and deletions. You read the diff and decide whether to accept, reject, or ask for changes, like reviewing a colleague's pull request.
Iterating / refinement prompt	Following up on the AI's first attempt with a correction ("good, but force the 3-part output structure"). Most real AI-assisted coding is a short back-and-forth, not one perfect prompt.
Validation check	A check you build in to confirm the AI's code is correct, e.g. printing 3 before/after examples to verify the PII filter actually redacted secrets. You prompt the AI to include these.
Graceful fallback	Code that does not crash when an API call fails. It records the failure per row instead, so the script always completes.

4. The business case: what you are building

IN PLAIN ENGLISH

CircleCI is a CI/CD platform: it automatically builds, tests, and deploys software every time a developer pushes code. When a build fails, the developer gets a wall of raw log output and has to read through it to work out what went wrong.

Right now there is nothing between that raw log and the developer. Common, trivially fixable errors demand the same manual reading as rare, complex ones. Your goal: build an intelligence layer that reads the log, classifies the failure, and explains it in plain English with a concrete fix, so the developer spends under 5 minutes, not 20, and does not open a support ticket.

Think of it like a check-engine light that does not just say "fault" but tells you "loose fuel cap, tighten it". You build that explanation layer by prompting Codex.

Business fact	Details
Platform	CircleCI, a cloud CI/CD platform; the dataset also covers GitHub Actions, GitLab CI, Jenkins, and Azure DevOps.
Current pain	Developers spend 18 to 22 minutes per failed build manually parsing logs to find root cause.
Ticket load	About 34% of support tickets are diagnostic, answerable from the log alone.
The target	Cut time-to-diagnosis to under 5 minutes and cut diagnostic tickets by at least 40% within 6 months.
The constraint	Do not change how developers interact with the platform.
Dataset	ci_cd_pipeline_failure_logs_dataset.csv, a real-world-inspired CI/CD failure log dataset, used as a public proxy for CircleCI's real logs.

REFLECT

1. Why is "just search the log for the word ERROR" not enough? What does a developer actually need beyond locating the error line?
2. Why is this an NLP problem and not a simple rules problem (e.g. a lookup table of known error strings)?
3. If 34% of tickets are diagnostic, what would a 40% reduction in those tickets be worth to the support team?

5. The AI-assisted coding workflow

The error summariser is the vehicle. The skill you are actually learning is how to build software by directing an AI agent and critically reviewing its work, and, specific to this module, how the wording of a prompt changes the quality of an NLP result. This is the through-line that connects all seven stages, so spend real time here.

5.1 The Generate, Review, Refine loop

IN PLAIN ENGLISH

Codex CLI is an agentic coding tool: you give it a task in plain language and it reads your files, writes or edits code, runs commands, and reports back. You direct and review; it does the typing and running.

Every stage follows the same three-step loop. Learn it in Stage 1, then name it to yourself each time so the rhythm becomes automatic.

GENERATE: You write a clear prompt stating the goal, context, and constraints. Codex proposes and often runs the code.

REVIEW: You read what it produced. Does it do what you asked? Is the prompt producing a useful NLP result, or a plausible-looking but wrong one? This is the step beginners skip and professionals never do.

REFINE: If it is off, you do not hand-fix. You tell Codex what to change, or you change the prompt. One or two follow-ups usually get there. If it is right, you accept.

THE GOLDEN RULE

You own every line the AI writes, and every prompt you ship. If you cannot explain why it works, you cannot ship it.

5.2 Anatomy of a good prompt

A weak prompt ("classify the error") gives generic, often wrong output. A strong prompt has four parts, and this structure is the heart of the whole module:

GOAL: what you want. "Classify each build error into one of these eight categories."

CONTEXT: what the AI needs to know. "These are sanitised CircleCI logs; the clean_error_message column has already had secrets redacted."

CONSTRAINTS: the rules it must follow. "Return only the category label, nothing else. Use exactly these labels."

OUTPUT SHAPE: what you want back. "Respond in the format WHAT FAILED / ROOT CAUSE / SUGGESTED FIX, one sentence each."

WATCH OUT

The AI acts confidently whether the prompt is good or not. A vague prompt does not error; it returns a fluent, wrong-shaped, or inconsistent answer that looks fine until you check it against the label set or the required structure. State your constraints, and review even when the output reads well.

5.3 Giving the agent durable context

State the problem, the dataset, and the hard rules once in your first prompt, then refer back. Codex carries this context through the session, so later prompts do not need to repeat it. The hard rules for this module are:

CONTEXT TO STATE ONCE

```
# CircleCI Error Intelligence - context to state once
GOAL: Read a failed-build log, classify it, and explain it in plain
English with a concrete fix. Cut time-to-diagnosis under 5 minutes.
DATA: ci_cd_pipeline_failure_logs_dataset.csv - CI/CD failure logs
across CircleCI, GitHub Actions, GitLab CI, Jenkins, Azure DevOps.
HARD RULES:
- Never send raw logs to the model - sanitise PII first.
- Classification must use ONLY the fixed label set.
- Summaries must follow the fixed output structure.
- Judge usefulness by task metrics, not word-overlap metrics.
```

Whether you keep this in a file or state it in your first message, the point is the same: encode the rules once so you are not re-typing them every prompt.

5.4 How to review AI-generated NLP code

This is the checklist to apply after every generation. Keep it beside you all the way through.

- Does it actually do what I asked, or something adjacent? AIs often answer a slightly different question.
- Label discipline: does the classifier only ever return labels from the fixed set, or does it invent new ones?
- Secret check: is every log sanitised before it leaves the machine? Did any key, token, or path survive?
- Metric check: is the evaluation metric appropriate for the task? Word-overlap metrics are wrong when there is no reference summary.
- Structure check: does the summary follow the required output shape every time, so it can be parsed?
- Can I explain every line and every prompt choice? If not, ask Codex to explain it before accepting. That is a valid prompt too.

Stage 1: Data exploration

Hand Codex the business brief and the data, let it profile the file, and verify the schema against what the prompt claims is there. The first lesson: never trust a spec over the data.

IN PLAIN ENGLISH

Before any NLP, you open the data and ask: what columns exist, what is the text we will work with, and what are we predicting? For this task the text is the `error_message` column and the labels live in `failure_type` and `failure_stage`.

Here is the twist: your prompt will claim the file has 28 columns. The real file has 25. A good agent profiles the data and tells you the truth instead of inventing the missing three. Your job is to confirm it did.

PROMPT 1: HAND OVER THE BRIEF AND ASK FOR GROUNDED EXPLORATION

I am working with CircleCI. When a build fails, developers get a raw log dump of 300-500 lines and spend 18-22 minutes finding the root cause; about 34% of support tickets are diagnostic. CircleCI wants time-to-diagnosis under 5 minutes and a 40% cut in diagnostic tickets. We have a failure logs dataset, `ci_cd_pipeline_failure_logs_dataset.csv`, with CI/CD execution and failure logs across CircleCI, GitHub Actions, GitLab CI, Jenkins, and Azure DevOps.

Create `explore.py` that loads it with pandas and prints: shape; all column names with dtypes; a 5-row sample of `ci_tool`, `failure_type`, `failure_stage`, `error_message`, `severity`, `pipeline_success`; unique counts of `failure_type` and `failure_stage`; and how many rows are CircleCI. Do not clean anything yet. Run it and show the output.

CHECK YOUR OUTPUT BEFORE YOU MOVE ON

Codex should report the true shape and flag the schema mismatch. Expect it to find 45,000 rows and 27 columns, not the 28 your prompt implied, and to note that `pipeline_success` is not actually present (it may add a placeholder so the sample view still renders). Confirm it surfaced this rather than silently inventing columns.

Confirm the basics on screen: 10 unique failure_type values, 3 unique failure_stage values, and about 9,007 CircleCI rows.

If Codex had silently produced 28 columns, that is your teaching moment to catch: the prompt said 28, the data has 25. Which do you trust? Always the data.

The screenshot shows the Codex IDE interface. The main editor area contains a large, light gray watermark of the Codex logo. Below the watermark, there are three keyboard shortcuts: "Open Chat" (Ctrl + Alt + I), "Show All Commands" (Ctrl + Shift + P), and "Find in Files" (Ctrl + Shift + F). On the right side, there is a chat window titled "CircleCI Error Intelligence". The chat content includes a goal to read a failed-build log, classify it, and explain it in plain English. It also lists data sources (ci_cd_pipeline_failure_logs_dataset.csv) and hard rules for the model, such as never sending raw logs and using only the fixed label set. At the bottom of the chat window, there is a button to "Approve for me" and a "Work locally" option.

The screenshot shows the Codex IDE interface with a Python script named "explore.py" open in the editor. The script is a data exploration tool that uses pandas to read a CSV file and display the results in a formatted way. The script includes comments and docstrings. The terminal at the bottom shows the output of the script, which is a DataFrame with 45,000 rows and 27 columns. The columns and their data types are listed. The chat window on the right shows the "CircleCI Error Intelligence" goal and the "Configuration error: 44v1" message. The terminal also shows the command "python .\explore.py" being executed.

```

1 from __future__ import annotations
2
3 from pathlib import Path
4 import re
5 from collections import Counter
6 import csv
7
8 try:
9     import pandas as pd # type: ignore
10 except ModuleNotFoundError: # pragma: no cover
11     pd = None # type: ignore
12
13
14 def _redact_for_display(value: object) -> object:
15     if not isinstance(value, str) or not value:
16         return value
17
18     text = value
19
20     # Emails
21     text = re.sub(r"\b[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}\b", "<REDACTED_EMAIL>", text, flags=re.IGNORECASE)
22     # IPv4
23     text = re.sub(r"\b(?:\d{1,3}\.){3}\d{1,3}\b", "<REDACTED_IP>", text)
24     # URLs
25     text = re.sub(r"\bhttps?://\S+\b", "<REDACTED_URL>", text, flags=re.IGNORECASE)
26     # Common secret/token-ish strings (conservative)
27     text = re.sub(r"\b[ghgtpoustr]_[A-Za-z0-9]{20,}\b", "<REDACTED_GH_TOKEN>", text) # GitHub tokens
28     text = re.sub(r"\bAKIA[0-9A-Z]{16}\b", "<REDACTED_AWS_KEY_ID>", text) # AWS access key id
29     text = re.sub(r"\b[A-Za-z0-9+/]{32,}\b", "<REDACTED_SECRET>", text) # long base64-ish blobs
30     text = re.sub(r"\b[0-9a-f]{32,}\b", "<REDACTED_HEX>", text, flags=re.IGNORECASE) # long hex strings

```

```

PS C:\Users\Desktop\AGAAI - Week 6 - Prompt Engineering for NLP Workflows\MLS\Codex> python .\explore.py
Dataset: ci_cd_pipeline_failure_logs.csv
NOTE: pandas is not installed in this environment; using csv fallback.

Shape: (45000, 27)

Columns + dtypes:
- pipeline_id: string (csv fallback)
- run_id: string (csv fallback)
- timestamp: string (csv fallback)
- ci_tool: string (csv fallback)
- repository: string (csv fallback)
- branch: string (csv fallback)

```

WHAT YOU SHOULD SEE

- ✓ Shape → (45000, 28); script runs with 0 errors
- ✓ Columns listed with dtypes; pipeline_success absent in the raw file (placeholder added so the sample renders)
- ✓ failure_type → 10 unique values; failure_stage → 3 unique values
- ✓ CircleCI rows → about 9,007
- ✓ error_message sample shows raw token-like strings after the word ERROR

WATCH OUT

An agent under instruction pressure can "helpfully" match a wrong spec. Watch for invented columns, invented value counts, or a sample that does not match the printed dtypes. The fix is always to read the actual printed output, not the prompt's assumptions.

KEY INSIGHT

Your prompt told Codex to expect 28 columns. The file has 27. A weaker agent would have hallucinated three columns to match your instruction. A good one checks the data and tells you the truth, which is exactly why you always verify.

REFLECT

4. The prompt said 28 columns; the data had 25. Why is it dangerous when an AI "corrects" the data to match your instruction instead of telling you you were wrong?
5. Which column is the actual NLP input, and which columns are the labels you will compare against?

Stage 2: PII filtering

Prompt Codex to scrub secrets out of every log before it is ever sent to a model. This is the first place a constraint in your prompt directly protects the business.

IN PLAIN ENGLISH

PII is Personally Identifiable Information, and in CI/CD logs it means API keys, auth tokens, file paths with usernames, and environment variable values embedded inside error traces. Sending those to an external LLM API is a security and compliance violation.

So the very first transformation in the pipeline is sanitisation: a function that redacts anything that looks like a secret and returns a clean string. Everything downstream works on the cleaned text, never the raw log.

PROMPT 2: SANITISE THE LOGS

The `error_message` column contains raw build log text. Before we send any of it to an LLM we must sanitise it: production logs contain API keys, tokens, file paths, and environment variable values.

Create `pii_filter.py` that loads the dataset and defines `sanitise_log(text)` which: removes long alphanumeric strings of 20+ chars (token-like); replaces absolute file paths with `[FILEPATH]`; replaces `KEY=VALUE` env assignments with `KEY=[REDACTED]`; returns the cleaned string. Apply it to `error_message` into a new `clean_error_message` column, print 3 before/after examples, and save `cleaned_logs.csv`. Run it and show the examples.

File Edit Selection View Go ...

EXPLORER

- CODEX
 - .env
 - ci_cd_pipeline_failure_logs.csv
 - explore.py
 - README.md

CHAT CODEX

CircleCI Error Intelligence GOAL: Read a failed-build log...

CircleCI rows: 9007

Worked for 1m 34s

Metric	Value
Tokens quota	1,142,242/100,000,000 used, 98,857,758 remaining
Resets in	5d 54m 3s
Total consumed	5,172,581 tokens, \$1.339135 spent
Request tokens	21,114 in / 1,318 out (479 reasoning)

Edited explore.py
+179 -0

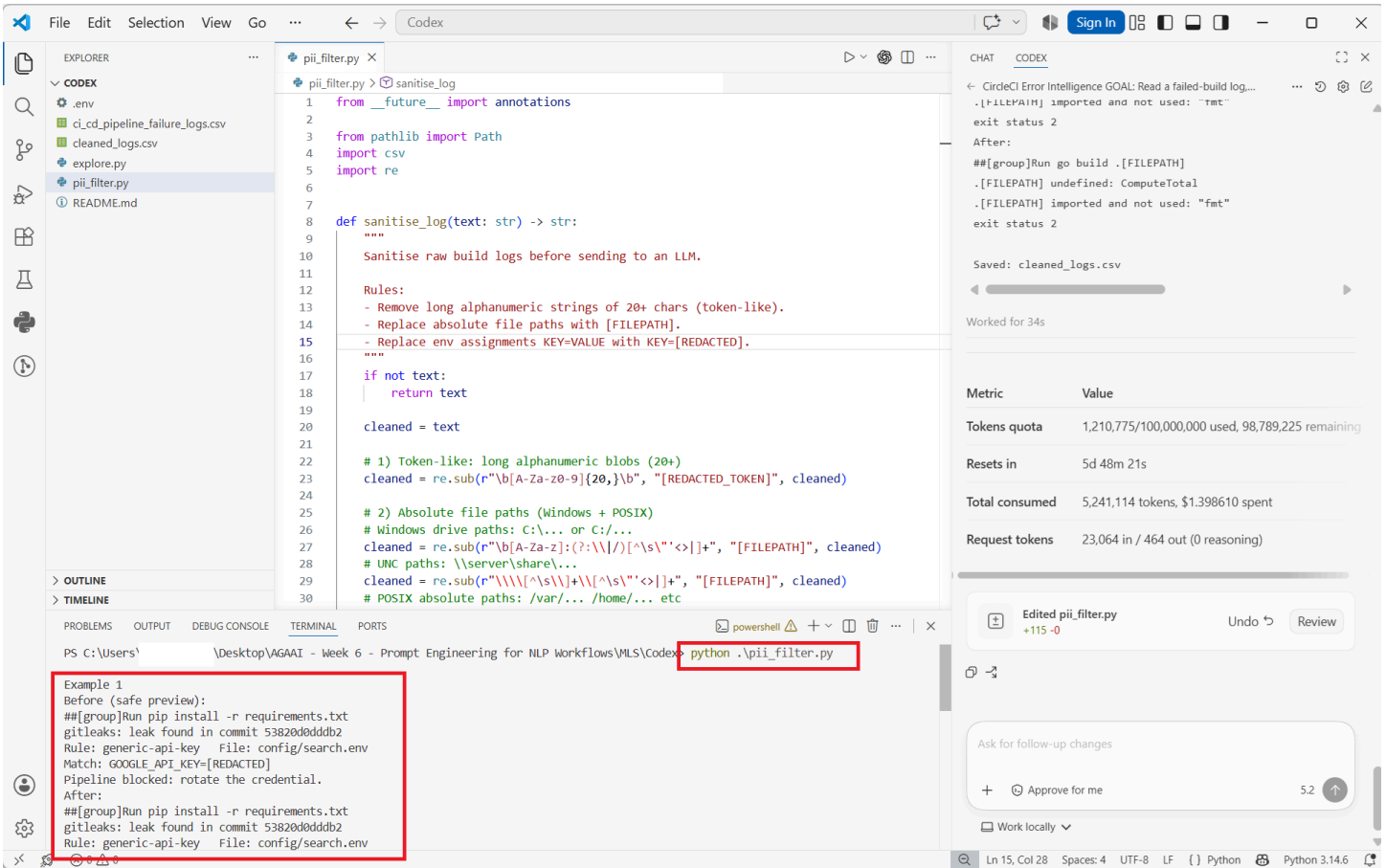
Undo Review

The error_message column contains raw build log text. Before we send any of it to an LLM we must sanitise it: production logs contain API keys, tokens, file paths, and environment variable values. Create pii_filter.py that loads the dataset and defines sanitise_log(text) which: removes long alphanumeric strings of 20+ chars (token-like); replaces absolute file paths with [FILEPATH]; replaces KEY=VALUE env assignments with KEY=[REDACTED]; returns the cleaned string. Apply it to error_message into a new clean_error_message column, print 3 before/after examples, and save cleaned_logs.csv. Run it and show the examples.

+ Approve for me 5.2

Work locally

Open Chat **Ctrl** + **Alt** + **I**
Show All Commands **Ctrl** + **Shift** + **P**
Find in Files **Ctrl** + **Shift** + **F**



The screenshot shows the VS Code editor with the file `pii_filter.py` open. The script defines a `sanitise_log` function that processes build logs. It includes rules for removing long alphanumeric strings, replacing absolute file paths with `[FILEPATH]`, and replacing environment variable assignments with `KEY=[REDACTED]`. The terminal output shows the command `python .\pii_filter.py` being executed, and the resulting cleaned logs are displayed in the terminal window.

CHECK YOUR OUTPUT BEFORE YOU MOVE ON

Read the 3 before/after examples closely. Expect token-like strings after ERROR to be replaced with `[REDACTED_TOKEN]`, and rows that contain no secret to be left intact. Confirm a secret actually disappeared, not just that the script ran.

Confirm `cleaned_logs.csv` was written and now carries a `clean_error_message` column. Everything from here uses that column.

WHAT YOU SHOULD SEE

- ✓ 3 before/after examples where token-like strings become `[REDACTED_TOKEN]`
- ✓ `cleaned_logs.csv` created with the new `clean_error_message` column
- ✓ Non-secret text (ordinary words) left untouched in the after column

WATCH OUT

A redaction regex can be too greedy or too loose. Too greedy and you destroy the very error text the model needs; too loose and a real key slips through. That is why you print before/after examples as a built-in validation check. The review is not optional.

On this synthetic dataset the logs are mostly random token-like text, so almost everything gets redacted. Expect near-total redaction here; it is a quirk of the demo data, and it will hurt downstream classification in Stage 3. Note it now so the weak classifier results later make sense. In real logs the redaction is surgical, not total.

KEY INSIGHT

This is a prompt-engineering project, but your first prompt is not to the model at all. It is to Codex, to build the guardrail. You never send a customer's secret to a third-party API to find out it was a secret.

REFLECT

6. Why sanitise before the model call rather than after? What is the risk of doing it the other way round?
7. If the filter is too aggressive and removes the real error text, what happens to the quality of the summary later? Where is the trade-off?

Stage 3: Error classification, zero-shot vs few-shot

The marquee prompt-engineering contrast. Same task, same data, two prompts, and the wording is the only thing that changes.

IN PLAIN ENGLISH

Classification means sorting each failure into one of a fixed set of categories so the right summary template can be chosen later. The NLP lesson here is about HOW you ask.

Zero-shot: you give only the instruction and the label list. Few-shot: you add a few worked examples first. The same model, on the same logs, can behave very differently depending on which you use, and that difference is the entire point of the module.

PROMPT 3A: ZERO-SHOT BASELINE

We have `cleaned_logs.csv` with a `clean_error_message` column. We need to classify each failure so we can pick the right summary template later.

Create `classifier.py` with `classify_error(error_message)` that calls the model and classifies into exactly one of: `dependency_resolution_failure`, `test_assertion_failure`, `misconfigured_script`, `environment_variable_missing`, `network_timeout`, `permission_denied`, `docker_build_failure`, `unknown`. Return only the label as a plain string. Run on 5 sample rows and print `error_message`, `predicted_label`, and the actual `failure_type`. Show output.

Metric	Value
Tokens quota	1,443,289/100,000,000 used, 98,556,711 remaining
Resets in	5d 44m 8s
Total consumed	5,473,628 tokens, \$1.549154 spent
Request tokens	31,985 in / 490 out (0 reasoning)

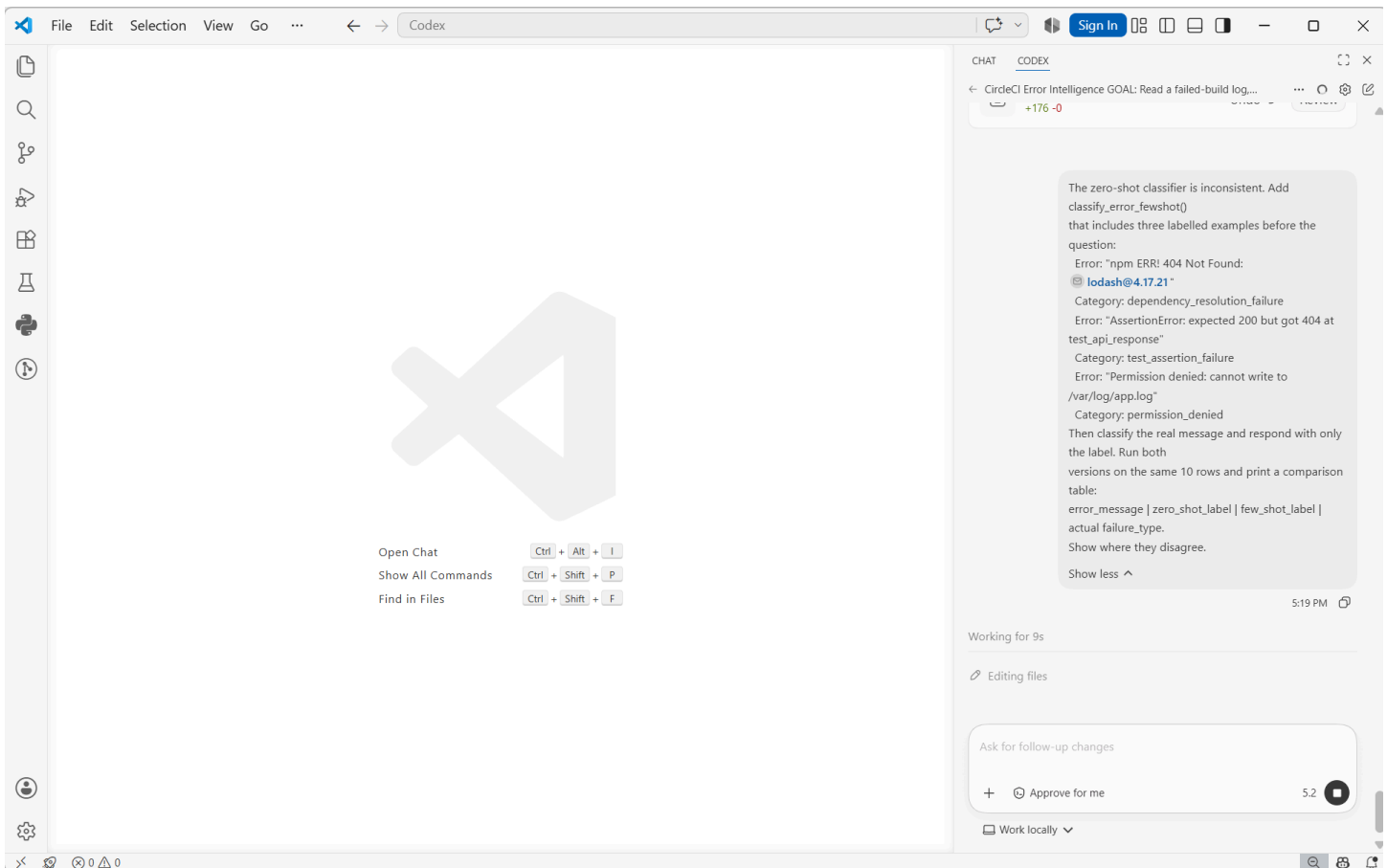
PROMPT 3B: FEW-SHOT UPGRADE

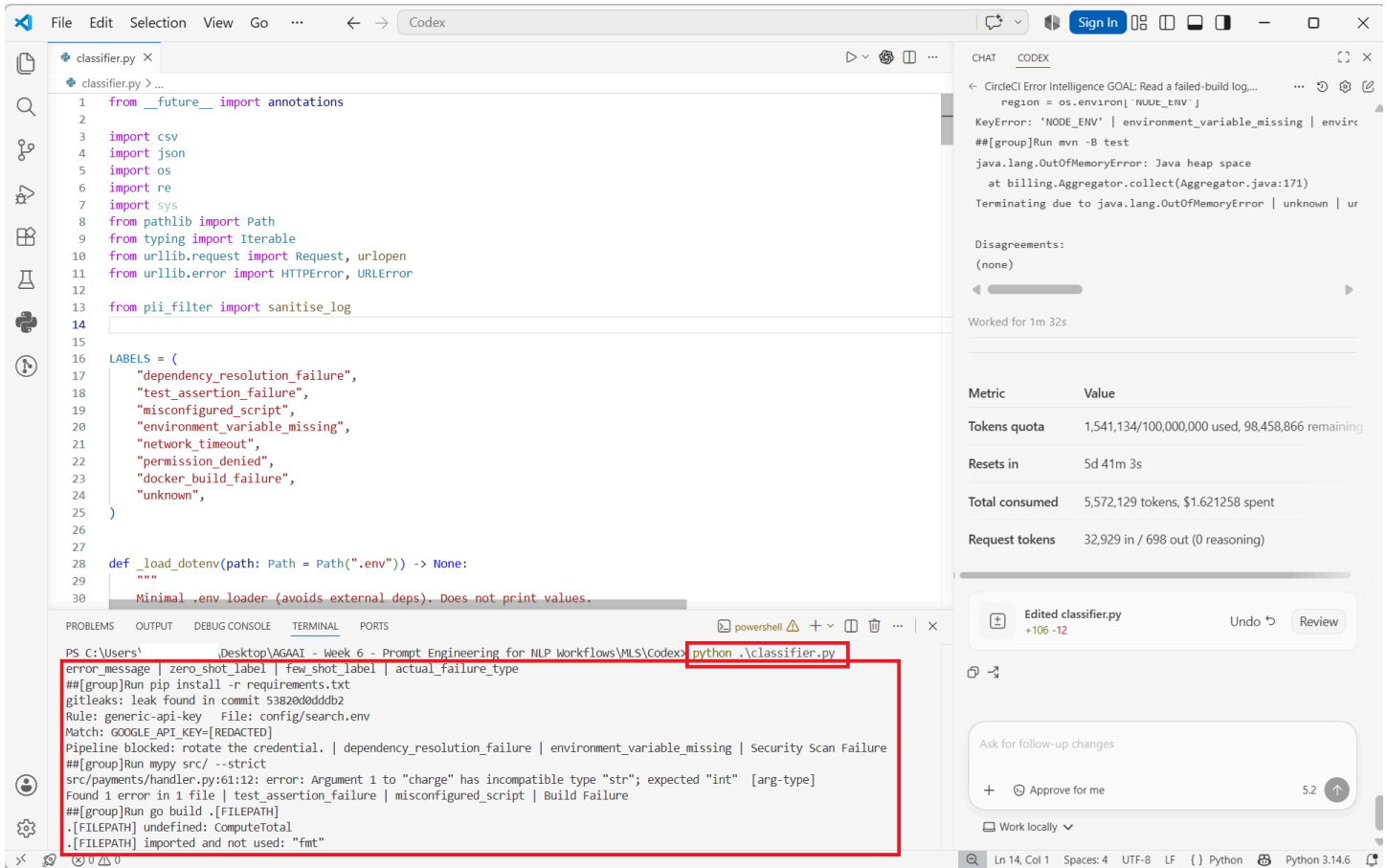
The zero-shot classifier is inconsistent. Add `classify_error_fewshot()` that includes three labelled examples before the question:

```
Error: "npm ERR! 404 Not Found: lodash@4.17.21"
Category: dependency_resolution_failure
Error: "AssertionError: expected 200 but got 404 at test_api_response"
Category: test_assertion_failure
Error: "Permission denied: cannot write to /var/log/app.log"
Category: permission_denied
```

Then classify the real message and respond with only the label. Run both versions on the same 10 rows and print a comparison table:

```
error_message | zero_shot_label | few_shot_label | actual failure_type.
Show where they disagree.
```





```

1 from __future__ import annotations
2
3 import csv
4 import json
5 import os
6 import re
7 import sys
8 from pathlib import Path
9 from typing import Iterable
10 from urllib.request import Request, urlopen
11 from urllib.error import HTTPError, URLError
12
13 from pii_filter import sanitise_log
14
15
16 LABELS = (
17     "dependency_resolution_failure",
18     "test_assertion_failure",
19     "misconfigured_script",
20     "environment_variable_missing",
21     "network_timeout",
22     "permission_denied",
23     "docker_build_failure",
24     "unknown",
25 )
26
27
28 def _load_dotenv(path: Path = Path(".env")) -> None:
29     """
30     Minimal .env loader (avoids external deps). Does not print values.

```

```

PS C:\Users\...> python .\classifier.py
error_message | zero_shot_label | few_shot_label | actual_failure_type
##[group]Run pip install -r requirements.txt
gitleaks: leak found in commit 53820d0dddb2
Rule: generic-api-key File: config/search.env
Match: GOOGLE_API_KEY=[REDACTED]
Pipeline blocked: rotate the credential. | dependency_resolution_failure | environment_variable_missing | Security Scan Failure
##[group]Run mypy src/ --strict
src/payments/handler.py:61:12: error: Argument 1 to "charge" has incompatible type "str"; expected "int" [arg-type]
Found 1 error in 1 file | test_assertion_failure | misconfigured_script | Build Failure
##[group]Run go build .[FILEPATH]
.[FILEPATH] undefined: ComputeTotal
.[FILEPATH] imported and not used: "fmt"

```

CHECK YOUR OUTPUT BEFORE YOU MOVE ON

Confirm both functions only ever return labels from the fixed set. The big review question: are the predictions actually differentiated, or is everything collapsing to one label?

Reality check on this dataset: because Stage 2 redacted almost all of the error text, both classifiers will frequently return unknown, so there may be no disagreements to show. That is not a bug in the prompt; it is the data quality flowing downstream. A great prompt cannot classify text that has been redacted away.

WHAT YOU SHOULD SEE

- ✓ Two functions, both constrained to the fixed label set
- ✓ A side-by-side comparison table across 10 rows
- ✓ On this dataset: many unknown labels and few or no disagreements, because the inputs were heavily redacted in Stage 2
- ✓ If you swap in logs with surviving error text, few-shot should anchor the labels more consistently than zero-shot

WATCH OUT

If the API key or model is misconfigured, the classifier silently falls back to unknown for every row, which can look like a prompt problem but is really an environment problem. Check that real model calls are succeeding before blaming the prompt. An unset key produces exactly this all-unknown pattern.

Few-shot can still be fooled by ambiguous logs, multi-cause failures in one message, or errors that resemble none of the examples. More or better examples is the refinement lever.

KEY INSIGHT

Examples anchor the model to your exact label schema and show it what a valid mapping looks like, which cuts the drift you see in zero-shot. You are teaching by demonstration inside the prompt. But prompt engineering sits downstream of data quality: fix the redaction, and the few-shot prompt starts to separate the labels.

REFLECT

8. Both versions returned mostly unknown here. How would you tell whether that is the prompt's fault, the data's fault, or the environment's fault?
9. Why does adding three examples change the answer at all, when the model and the input are identical?
10. What kind of error would still fool the few-shot classifier even on clean data?

Stage 4: Summarisation with LangChain, basic vs persona + output spec

The text-to-text task. You turn a cryptic failure into a plain-English explanation, then sharpen it with two prompt-engineering techniques and watch the output change.

IN PLAIN ENGLISH

Summarisation here is abstractive: the model writes a new explanation rather than copying log lines. You build the prompt as a reusable LangChain PromptTemplate with blanks for ci_tool, failure_stage, the cleaned error, and severity.

A first, plain template produces something generic. Then two techniques sharpen it: a persona ("you are a senior DevOps engineer") sets tone and depth, and an output specification forces a fixed, parseable structure: WHAT FAILED, ROOT CAUSE, SUGGESTED FIX.

PROMPT 4A: BASIC PROMPTTEMPLATE

```
We have cleaned_logs.csv with ci_tool, failure_stage, clean_error_message,
severity, failure_type. Create summariser.py that loads it, defines a
LangChain PromptTemplate called error_summary_prompt with variables
ci_tool, failure_stage, error_message, severity, asking the model to
explain the error in plain English for a developer who did not write the
failing code. Run it on 3 rows and print the FILLED prompt before each
response so we can see exactly what was sent. Show the output.
```


File Edit Selection View Go ...

EXPLORER

- CODEX
 - __pycache__
 - .env
 - ci_cd_pipeline_failure_logs.csv
 - classifier.py
 - cleaned_logs.csv
 - explore.py
 - pii_filter.py
 - README.md

Search

Run and Debug

Terminal

```
PS C:\Users\...> python .\summariser.py
```

Chat

CircleCI Error Intelligence GOAL: Read a failed-build log...

Metric	Value
Tokens quota	1,541,134/100,000,000 used, 98,458,866 remaining
Resets in	5d 41m 3s
Total consumed	5,572,129 tokens, \$1.621258 spent
Request tokens	32,929 in / 698 out (0 reasoning)

Edited classifier.py +106 -12

Undo Review

We have cleaned_logs.csv with ci_tool, failure_stage, clean_error_message, severity, failure_type. Create summariser.py that loads it, defines a LangChain PromptTemplate called error_summary_prompt with variables ci_tool, failure_stage, error_message, severity, asking the model to explain the error in plain English for a developer who did not write the failing code. Run it on 3 rows and print the FILLED prompt before each response so we can see exactly what was sent. Show the output.

Thinking

Ask for follow-up changes

+ Approve for me 5.2

Work locally

File Edit Selection View Go ...

EXPLORER

- CODEX
 - __pycache__
 - .env
 - ci_cd_pipeline_failure_logs.csv
 - classifier.py
 - cleaned_logs.csv
 - explore.py
 - pii_filter.py
 - README.md
 - summariser.py

Search

Run and Debug

Terminal

```
PS C:\Users\...> python .\summariser.py
```

Chat

CircleCI Error Intelligence GOAL: Read a failed-build log...

Respond with:

- 1) What happened (1-2 sentences)
- 2) Likely root cause
- 3) Concrete fix (steps)
- 4) Quick verification (1 command)

Row 3: MODEL RESPONSE

(model call failed: curl error [MinError 10061] No conn

Worked for 49s

Metric	Value
Tokens quota	1,686,559/100,000,000 used, 98,313,441 remaining
Resets in	5d 38m 23s
Total consumed	5,720,825 tokens, \$1.704046 spent
Request tokens	37,205 in / 722 out (0 reasoning)

Edited summariser.py +137 -0

Undo Review

Ask for follow-up changes

+ Approve for me 5.2

Work locally

Ln 10, Col 1 Spaces: 4 UTF-8 LF Python Python 3.14.6

PROMPT 4B: ADD PERSONA AND OUTPUT SPECIFICATION

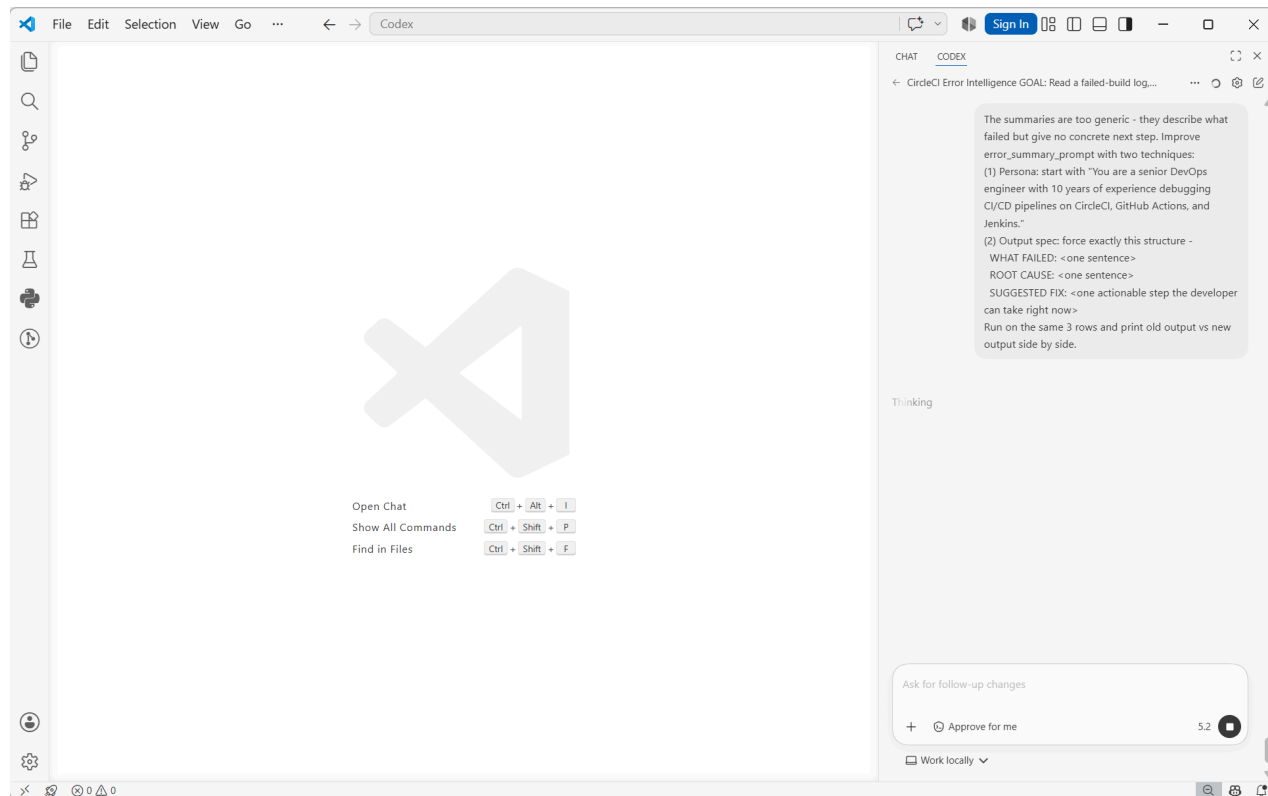
The summaries are too generic - they describe what failed but give no concrete next step. Improve error_summary_prompt with two techniques:
(1) Persona: start with "You are a senior DevOps engineer with 10 years of experience debugging CI/CD pipelines on CircleCI, GitHub Actions, and Jenkins."
(2) Output spec: force exactly this structure -

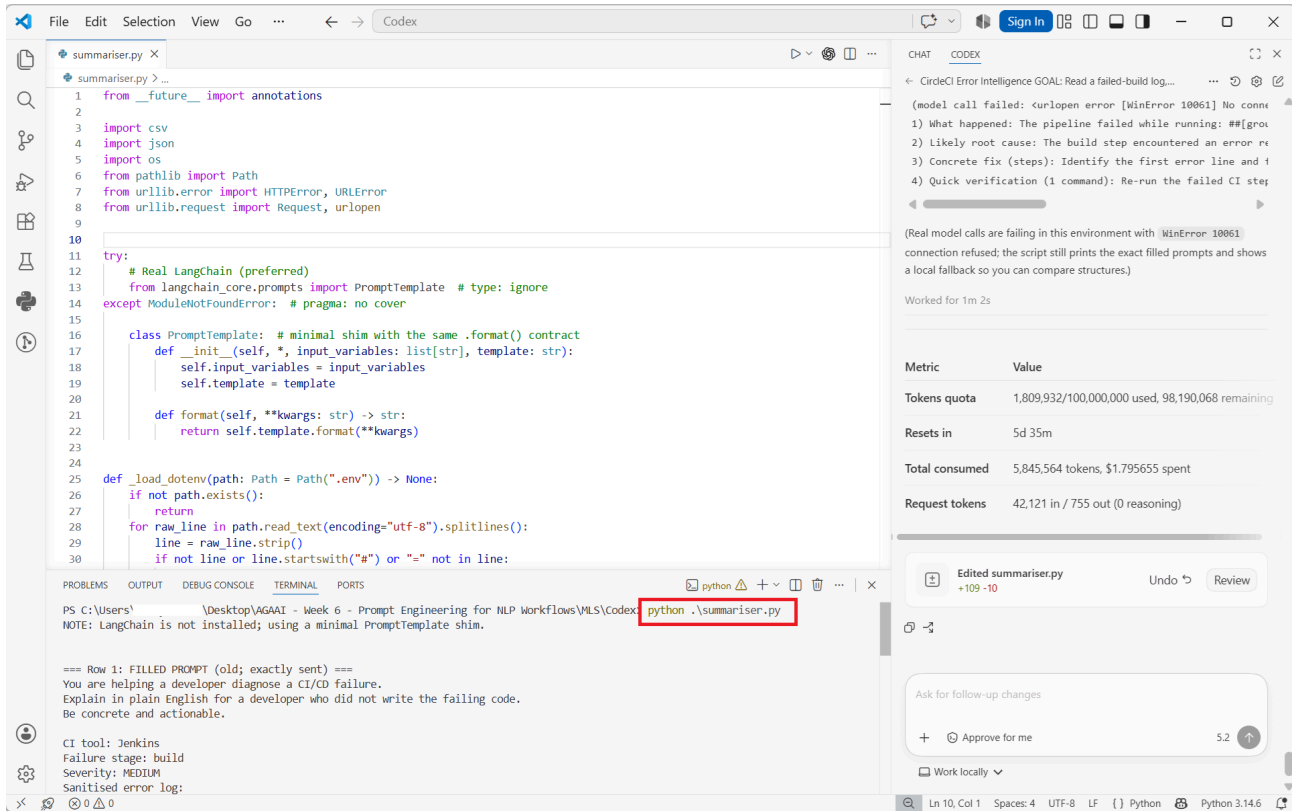
WHAT FAILED: <one sentence>

ROOT CAUSE: <one sentence>

SUGGESTED FIX: <one actionable step the developer can take right now>

Run on the same 3 rows and print old output vs new output side by side.





```

1 from __future__ import annotations
2
3 import csv
4 import json
5 import os
6 from pathlib import Path
7 from urllib.error import HTTPError, URLError
8 from urllib.request import Request, urlopen
9
10
11 try:
12     # Real LangChain (preferred)
13     from langchain_core.prompts import PromptTemplate # type: ignore
14 except ModuleNotFoundError: # pragma: no cover
15
16     class PromptTemplate: # minimal shim with the same .format() contract
17         def __init__(self, *, input_variables: list[str], template: str):
18             self.input_variables = input_variables
19             self.template = template
20
21         def format(self, **kwargs: str) -> str:
22             return self.template.format(**kwargs)
23
24
25 def _load_dotenv(path: Path = Path(".env")) -> None:
26     if not path.exists():
27         return
28     for raw_line in path.read_text(encoding="utf-8").splitlines():
29         line = raw_line.strip()
30         if not line or line.startswith("#") or "=" not in line:

```

PS C:\Users\... \Desktop\AGAAI - Week 6 - Prompt Engineering for NLP Workflows\MLS\Codex> python .\summariser.py

NOTE: Langchain is not installed; using a minimal PromptTemplate shim.

=== Row 1: FILLED PROMPT (old; exactly sent) ===
You are helping a developer diagnose a CI/CD failure.
Explain in plain English for a developer who did not write the failing code.
Be concrete and actionable.

CI tool: Jenkins
Failure stage: build
Severity: MEDIUM
Sanitised error log:

CHAT CODEX

CircleCI Error Intelligence GOAL: Read a failed-build log...

(model call failed: <urlopen error [WinError 10061] No connection...)

- 1) What happened: The pipeline failed while running: ##[gro...
- 2) Likely root cause: The build step encountered an error re...
- 3) Concrete fix (steps): Identify the first error line and f...
- 4) Quick verification (1 command): Re-run the failed CI step...

(Real model calls are failing in this environment with WinError 10061... connection refused; the script still prints the exact filled prompts and shows a local fallback so you can compare structures)

Worked for 1m 2s

Metric	Value
Tokens quota	1,809,932/100,000,000 used, 98,190,068 remaining
Resets in	5d 35m
Total consumed	5,845,564 tokens, \$1.795655 spent
Request tokens	42,121 in / 755 out (0 reasoning)

Edited summariser.py +109 -10 Undo Review

Ask for follow-up changes

+ Approve for me 5.2

Work locally

Ln 10, Col 1 Spaces: 4 UTF-8 LF {} Python Python 3.14.6

CHECK YOUR OUTPUT BEFORE YOU MOVE ON

Confirm the FILLED prompt is printed before each response. Seeing the exact text sent to the model is the single most useful thing in this whole module. It makes prompt engineering visible.

Confirm the upgraded version follows the three-part structure every time. If even one row breaks the format, the output spec is not strong enough yet. That is a refinement moment.

With the gateway and model correctly set, the upgraded prompt should produce structured, on-format summaries for all rows.

WHAT YOU SHOULD SEE

- ✓ The filled prompt printed before each model response
- ✓ Basic version: fluent but vague explanations, no fixed structure
- ✓ Upgraded version: every row in WHAT FAILED / ROOT CAUSE / SUGGESTED FIX form
- ✓ A side-by-side comparison making the difference obvious

WATCH OUT

Without an explicit output spec, the model drifts in format from row to row, which breaks any code that tries to parse WHAT FAILED later (you need exactly that in Stage 6). Force the structure and verify it holds on every row.

A persona can over-steer: too strong a persona and the model invents confident detail the log does not support. Keep the persona about tone and seniority, not about facts.

KEY INSIGHT

Same model, same error, two small additions, a role and a required shape, and the output goes from a paragraph nobody can act on to a fix a developer can run. That is prompt engineering in one screen.

REFLECT

11. Why does printing the filled prompt matter so much? What can you only debug once you can see it?
12. Which of the two changes, persona or output spec, did more of the work here, and why?
13. The output spec makes the summary parseable. Why will that matter in the evaluation stage?

Stage 5: Advanced prompting, Chain-of-Thought and self-consistency

These patterns trade speed and cost for reliability on the hard cases. Self-consistency makes five model calls per row, so run it once to see the mechanism rather than repeating it.

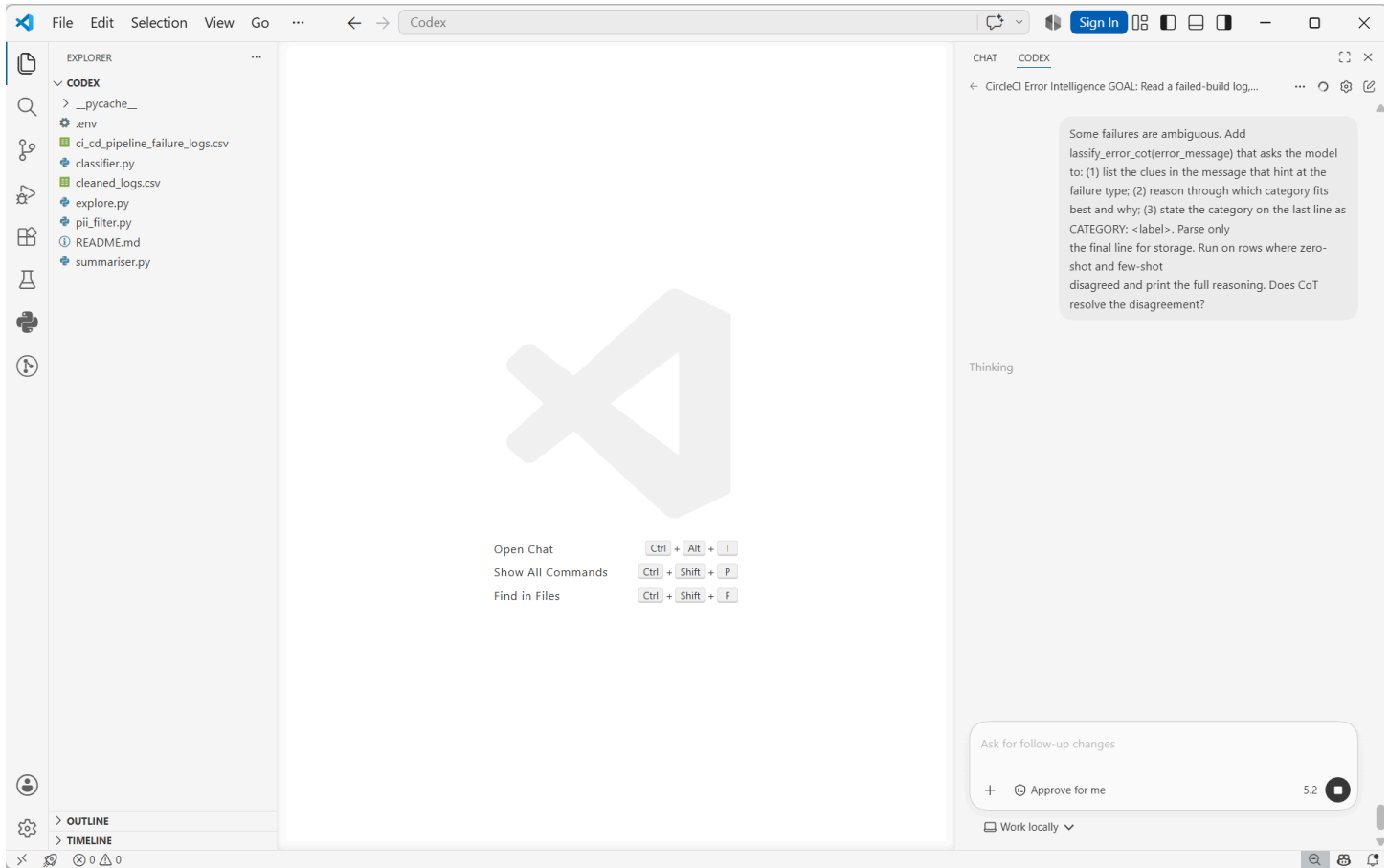
IN PLAIN ENGLISH

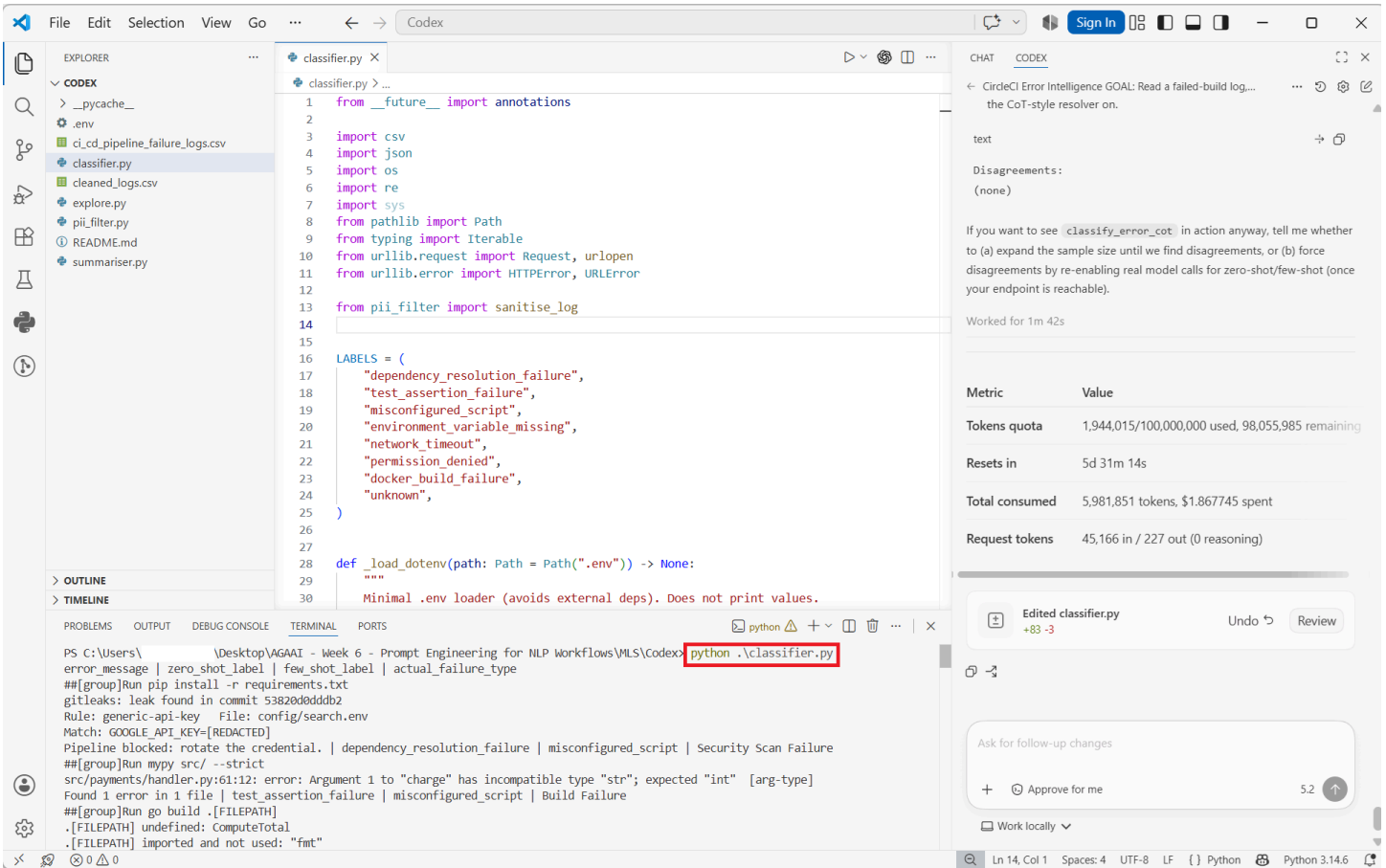
Some failures are genuinely ambiguous. A "connection refused" could be a network timeout, a wrong endpoint, or a missing environment variable. A model that pattern-matches the surface text mislabels these.

Chain-of-Thought asks the model to reason step by step before naming a category. Self-consistency runs that reasoning several times and takes a majority vote, so one unlucky run does not decide the label. Together they trade speed and cost for reliability on the hard cases.

PROMPT 5A: CHAIN-OF-THOUGHT

Some failures are ambiguous. Add `classify_error_cot(error_message)` that asks the model to: (1) list the clues in the message that hint at the failure type; (2) reason through which category fits best and why; (3) state the category on the last line as `CATEGORY: <label>`. Parse only the final line for storage. Run on rows where zero-shot and few-shot disagreed and print the full reasoning. Does CoT resolve the disagreement?





EXPLORER

- CODEX
 - __pycache__
 - .env
 - ci_cd_pipeline_failure_logs.csv
 - classifier.py
 - cleaned_logs.csv
 - explore.py
 - pii_filter.py
 - README.md
 - summariser.py

classifier.py

```

1  from __future__ import annotations
2
3  import csv
4  import json
5  import os
6  import re
7  import sys
8  from pathlib import Path
9  from typing import Iterable
10 from urllib.request import Request, urlopen
11 from urllib.error import HTTPError, URLError
12
13 from pii_filter import sanitise_log
14
15
16 LABELS = (
17     "dependency_resolution_failure",
18     "test_assertion_failure",
19     "misconfigured_script",
20     "environment_variable_missing",
21     "network_timeout",
22     "permission_denied",
23     "docker_build_failure",
24     "unknown",
25 )
26
27
28 def _load_dotenv(path: Path = Path(".env")) -> None:
29     """
30     Minimal .env loader (avoids external deps). Does not print values.

```

TERMINAL

```

PS C:\Users\... \Desktop\AGAAI - Week 6 - Prompt Engineering for NLP Workflows\MLS\Codex> python .\classifier.py
error_message | zero_shot_label | few_shot_label | actual_failure_type
##[group]Run pip install -r requirements.txt
gitleaks: leak found in commit 53820d0dddb2
Rule: generic-api-key File: config/search.env
Match: GOOGLE_API_KEY=[REDACTED]
Pipeline blocked: rotate the credential. | dependency_resolution_failure | misconfigured_script | Security Scan Failure
##[group]Run mypy src/ --strict
src/payments/handler.py:61:12: error: Argument 1 to "charge" has incompatible type "str"; expected "int" [arg-type]
Found 1 error in 1 file | test_assertion_failure | misconfigured_script | Build Failure
##[group]Run go build .[FILEPATH]
.[FILEPATH] undefined: ComputeTotal
.[FILEPATH] imported and not used: "fmt"

```

CHAT CODEX

CircleCI Error Intelligence GOAL: Read a failed-build log... the CoT-style resolver on.

text

Disagreements: (none)

If you want to see `classify_error_cot` in action anyway, tell me whether to (a) expand the sample size until we find disagreements, or (b) force disagreements by re-enabling real model calls for zero-shot/few-shot (once your endpoint is reachable).

Worked for 1m 42s

Metric	Value
Tokens quota	1,944,015/100,000,000 used, 98,055,985 remaining
Resets in	5d 31m 14s
Total consumed	5,981,851 tokens, \$1.867745 spent
Request tokens	45,166 in / 227 out (0 reasoning)

Edited classifier.py +83 -3 Undo Review

Ask for follow-up changes

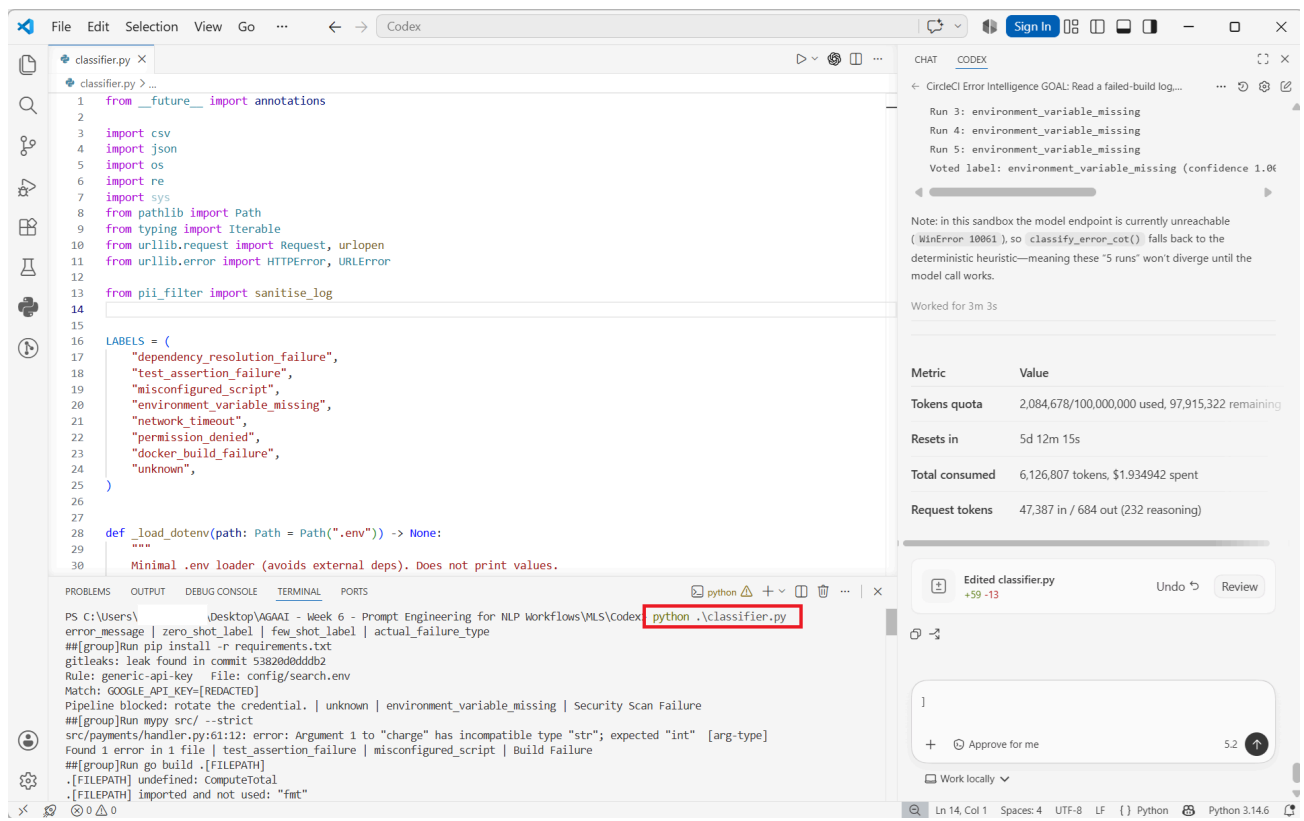
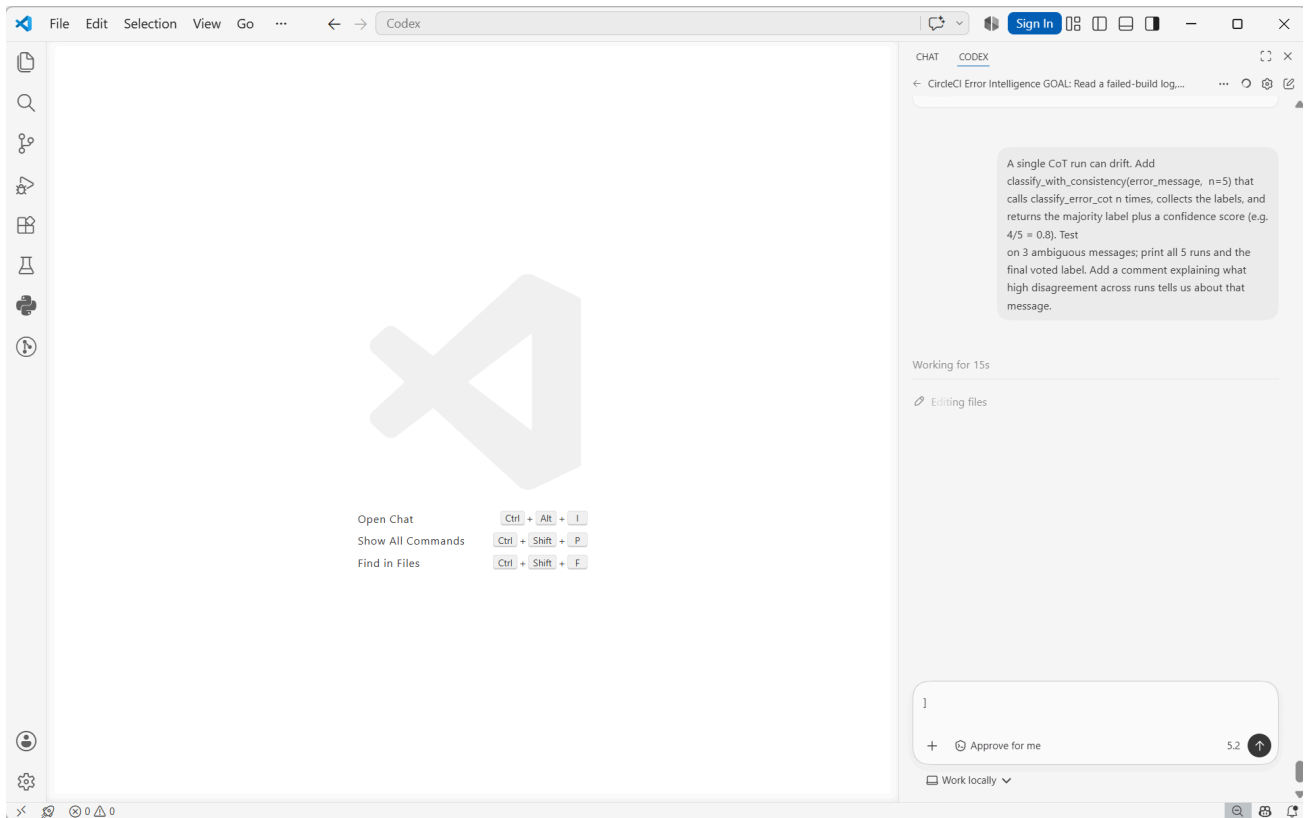
+ Approve for me 5.2

Work locally

Ln 14, Col 1 Spaces: 4 UTF-8 LF {} Python Python 3.14.6

PROMPT 5B: SELF-CONSISTENCY

A single CoT run can drift. Add `classify_with_consistency(error_message, n=5)` that calls `classify_error_cot` `n` times, collects the labels, and returns the majority label plus a confidence score (e.g. $4/5 = 0.8$). Test on 3 ambiguous messages; print all 5 runs and the final voted label. Add a comment explaining what high disagreement across runs tells us about that message.



CHECK YOUR OUTPUT BEFORE YOU MOVE ON

Confirm the final-line parser extracts only the label after CATEGORY: and ignores the reasoning above it. A common bug is storing the whole reasoning blob as the label.

Confirm the vote is a real majority over n runs and that confidence is votes-for-winner divided by n .

Honesty check: on the redacted synthetic data these runs also return unknown, so confidence is high but the label is uninformative. The mechanism is correct; the input has no signal left. High agreement on no information is not the same as a good answer.

WHAT YOU SHOULD SEE

- ✓ CoT output showing clues, reasoning, then a clean CATEGORY: line
- ✓ Five individual runs per message and a majority-voted label with a confidence score
- ✓ On clean data: disagreement on ambiguous cases that the vote then resolves
- ✓ On this redacted dataset: consistent unknown with high vote confidence but no real signal

WATCH OUT

Self-consistency can agree five times on unknown. High agreement on no information is still no information. Confidence is not correctness.

CoT and self-consistency multiply cost. Chain-of-Thought is one model call per row; self-consistency is five. Reserve them for the ambiguous cases where the extra reliability is worth the cost.

KEY INSIGHT

You are letting the model think out loud before it commits, then letting it check itself with a vote. Both buy reliability on genuinely hard inputs, but neither can manufacture signal that the data does not contain.

REFLECT

14. Why can reasoning step by step beat answering immediately on an ambiguous log?
15. Self-consistency agreed five times on unknown. Why is high confidence here misleading?
16. When is the extra cost of five calls per row worth it, and when is it not?

Stage 6: Evaluation, when the standard metric is the wrong metric

The hardest and most valuable lesson in the module. You reach for the textbook summarisation metrics, watch them fail honestly, understand why, and pivot to metrics that actually measure the task.

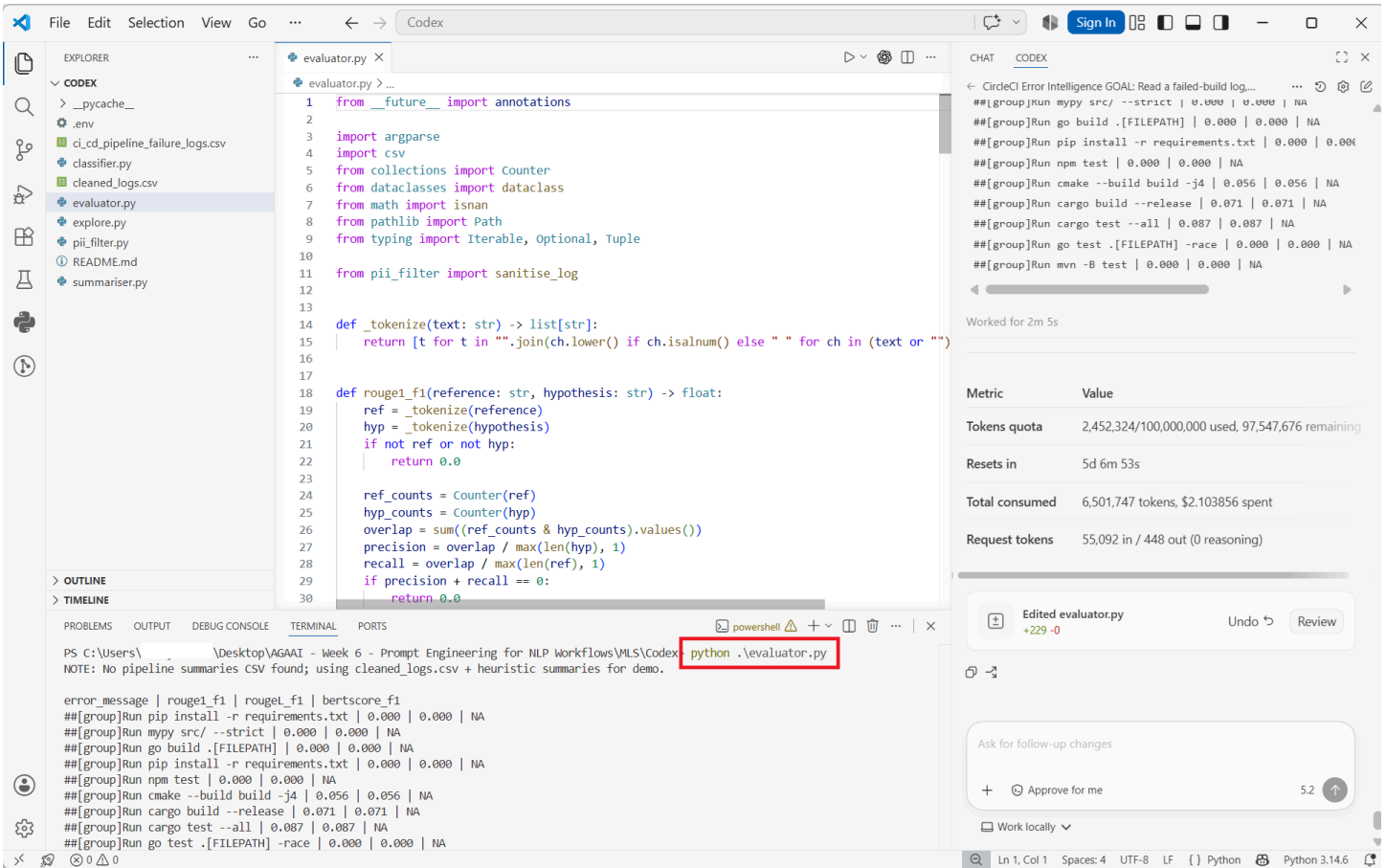
IN PLAIN ENGLISH

ROUGE and BERTScore compare a generated summary against a reference summary. ROUGE counts word overlap; BERTScore compares meaning via embeddings. Both assume you have a human-written reference to compare to.

You do not. Your only "reference" is the raw error log, which is random token text, not a model summary. So word-overlap scores collapse to zero and embedding scores come back as not-a-number. The right move is not to fix the numbers but to change the question: judge the summary by whether it is accurate and actionable, using an LLM-as-a-Judge rubric.

PROMPT 6A: THE TEXTBOOK METRICS (WHICH WILL FAIL HONESTLY)

```
Create evaluator.py that loads the pipeline summaries, defines
evaluate_summary(reference, hypothesis) returning ROUGE-1 F1, ROUGE-L F1,
and BERTScore F1 (distilbert-base-uncased), using original_error as
reference and the WHAT FAILED sentence as hypothesis. Print a table
error_message | rouge1 | rougeL | bertscore. Add a comment on why a low
ROUGE here does not mean a bad summary.
```



The screenshot displays the Codex IDE interface. The main editor shows a Python file named `evaluator.py` with the following code:

```
1 from __future__ import annotations
2
3 import argparse
4 import csv
5 from collections import Counter
6 from dataclasses import dataclass
7 from math import isnan
8 from pathlib import Path
9 from typing import Iterable, Optional, Tuple
10
11 from pii_filter import sanitise_log
12
13
14 def _tokenize(text: str) -> list[str]:
15     return [t for t in "".join(ch.lower() if ch.isalnum() else " " for ch in (text or ""))]
16
17
18 def rouge1_f1(reference: str, hypothesis: str) -> float:
19     ref = _tokenize(reference)
20     hyp = _tokenize(hypothesis)
21     if not ref or not hyp:
22         return 0.0
23
24     ref_counts = Counter(ref)
25     hyp_counts = Counter(hyp)
26     overlap = sum((ref_counts & hyp_counts).values())
27     precision = overlap / max(len(hyp), 1)
28     recall = overlap / max(len(ref), 1)
29     if precision + recall == 0:
30         return 0.0
```

The terminal window at the bottom shows the command `python .\evaluator.py` being executed, with the output:

```
PS C:\Users\...\Desktop\AGAAI - Week 6 - Prompt Engineering for NLP Workflows\MLS\Codex> python .\evaluator.py
NOTE: No pipeline summaries CSV found; using cleaned_logs.csv + heuristic summaries for demo.

error_message | rouge1_f1 | rouge1_f1 | bertscore_f1
##[group]Run pip install -r requirements.txt | 0.000 | 0.000 | NA
##[group]Run mypy src/ --strict | 0.000 | 0.000 | NA
##[group]Run go build .[FILEPATH] | 0.000 | 0.000 | NA
##[group]Run pip install -r requirements.txt | 0.000 | 0.000 | NA
##[group]Run npm test | 0.000 | 0.000 | NA
##[group]Run cmake --build build -j4 | 0.056 | 0.056 | NA
##[group]Run cargo build --release | 0.071 | 0.071 | NA
##[group]Run cargo test --all | 0.087 | 0.087 | NA
##[group]Run go test .[FILEPATH] -race | 0.000 | 0.000 | NA
```

The chat window on the right shows a conversation with the AI assistant, including a table of metrics:

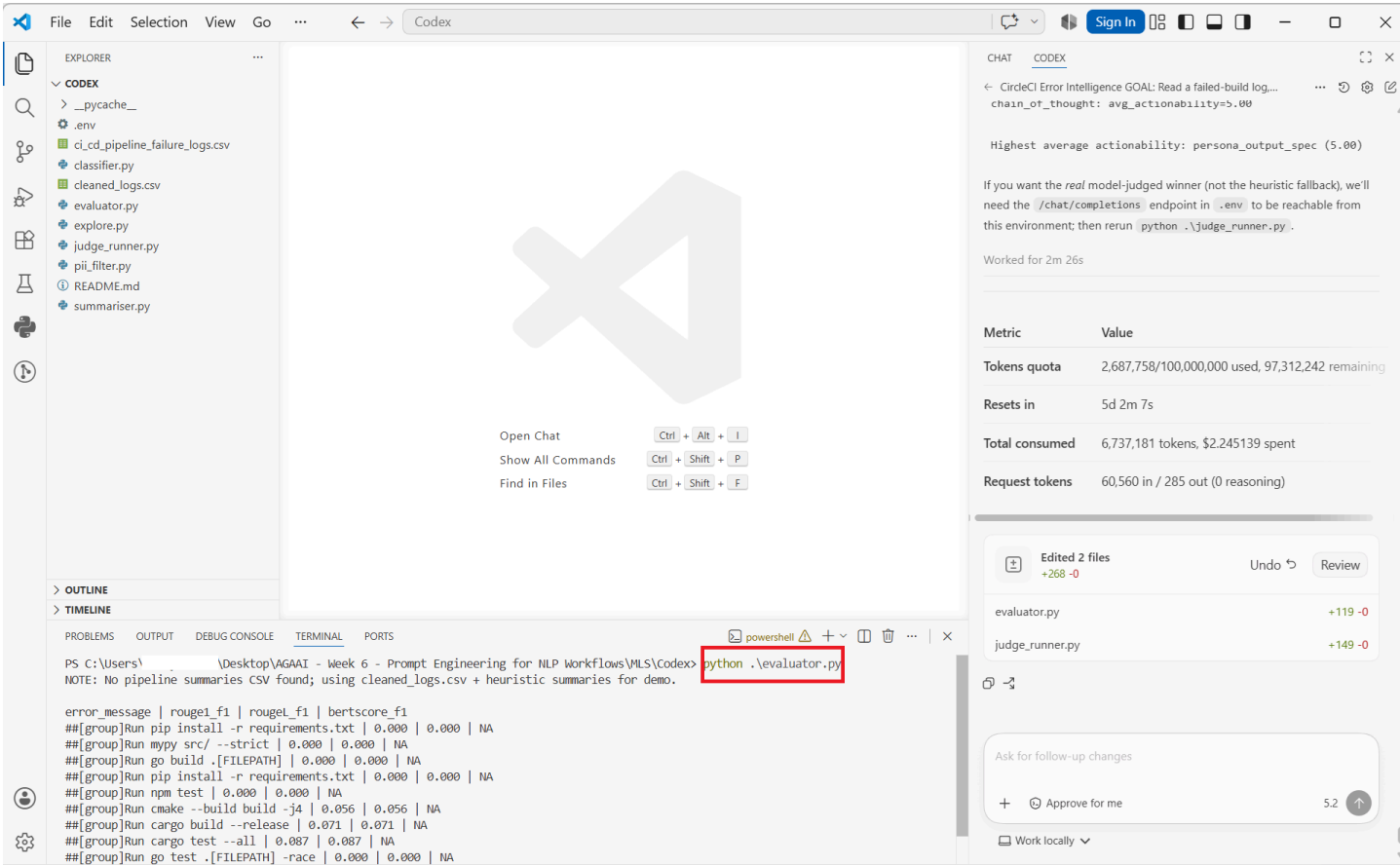
Metric	Value
Tokens quota	2,452,324/100,000,000 used, 97,547,676 remaining
Resets in	5d 6m 53s
Total consumed	6,501,747 tokens, \$2.103856 spent
Request tokens	55,092 in / 448 out (0 reasoning)

PROMPT 6B: PIVOT TO THE RIGHT METRICS (LLM-AS-A-JUDGE)

ROUGE and BERTScore measure similarity to the log text, but what we care about is whether a developer reading the summary knows what to do. Add `judge_summary(original_error, summary)` that asks the model:

"You are evaluating an AI-generated build error explanation for a CircleCI developer. Original error: {original_error} Generated summary: {summary}. Score 1-5 on Accuracy (does it correctly describe what went wrong) and Actionability (does the fix give something concrete to do now). Respond exactly:
 ACCURACY: <score>
 ACTIONABILITY: <score>
 REASON: <one sentence>"

Run it on all 5 summaries across three techniques - basic template, persona+output spec, and Chain-of-Thought - and report which technique gave the highest average actionability.



Open Chat: `Ctrl + Alt + I`
 Show All Commands: `Ctrl + Shift + P`
 Find in Files: `Ctrl + Shift + F`

PS C:\Users\... \Desktop\AGAAI - Week 6 - Prompt Engineering for NLP Workflows\MLS\Codex> `python .\evaluator.py`

NOTE: No pipeline summaries CSV found; using cleaned_logs.csv + heuristic summaries for demo.

Metric	Value
Tokens quota	2,687,758/100,000,000 used, 97,312,242 remaining
Resets in	5d 2m 7s
Total consumed	6,737,181 tokens, \$2.245139 spent
Request tokens	60,560 in / 285 out (0 reasoning)

Edited 2 files
 +268 -0

evaluator.py +119 -0
 judge_runner.py +149 -0

Ask for follow-up changes
 + Approve for me 5.2

CHECK YOUR OUTPUT BEFORE YOU MOVE ON

Expect ROUGE-1 and ROUGE-L to come back 0.0 for every row and BERTScore to come back nan. This is the teaching moment, not a failure, and there are two distinct causes: ROUGE is 0.0 because the raw token-like reference shares almost no words with a natural-language WHAT FAILED sentence (nothing to overlap); BERTScore is nan because the heavy embedding runtime (torch and transformers) may not be fully available, so the metric cannot execute.

Do not paper over this with a fake number. The honest reading: the metric does not fit the task.

For 6B, confirm the judge parses ACCURACY and ACTIONABILITY as numbers and records the one-sentence reason, and that it scored all three techniques on the same rows so the comparison is fair.

WHAT YOU SHOULD SEE

- ✓ ROUGE table of zeros and nan from 6A, kept on screen as the lesson, not hidden
- ✓ A working LLM-as-a-Judge giving ACCURACY, ACTIONABILITY, and a REASON per summary
- ✓ Average actionability: basic template 1.2, persona+output spec 2.0, CoT 2.0
- ✓ A clear result: the engineered prompts beat the basic one on the metric that matters

WATCH OUT

Asked to "evaluate the summaries", Codex will reach for ROUGE/BERTScore by default because they are the textbook summarisation metrics, and will report zeros and nan without flagging that the metric does not fit. You must catch that the metric is wrong, not just that the score is low.

An LLM judge needs a strict output format or its scores cannot be parsed. Pin the format exactly, as in 6B, and verify the parser.

KEY INSIGHT

Choosing the right metric is itself a skill. The most dangerous evaluation is a confident number that measures the wrong thing. Once you switch to accuracy and actionability, the prompt work from Stages 4 and 5 pays off in a measurable jump.

REFLECT

17. ROUGE was 0 and BERTScore was nan. For each, was the cause the metric being wrong for the task, or a technical problem? (ROUGE: wrong-for-task. BERTScore: a runtime problem.)
18. Why is "similarity to the original log" the wrong thing to measure for this product?
19. The basic template scored 1.2 and the engineered prompts 2.0. What does that prove about the value of prompt engineering?

Stage 7: End-to-end pipeline and the feedback loop

Wire every stage into one pipeline, find the weakest summaries by their judge score, and use that to improve the prompt. Closing the loop from evaluation back to prompt design is the defining move of prompt engineering.

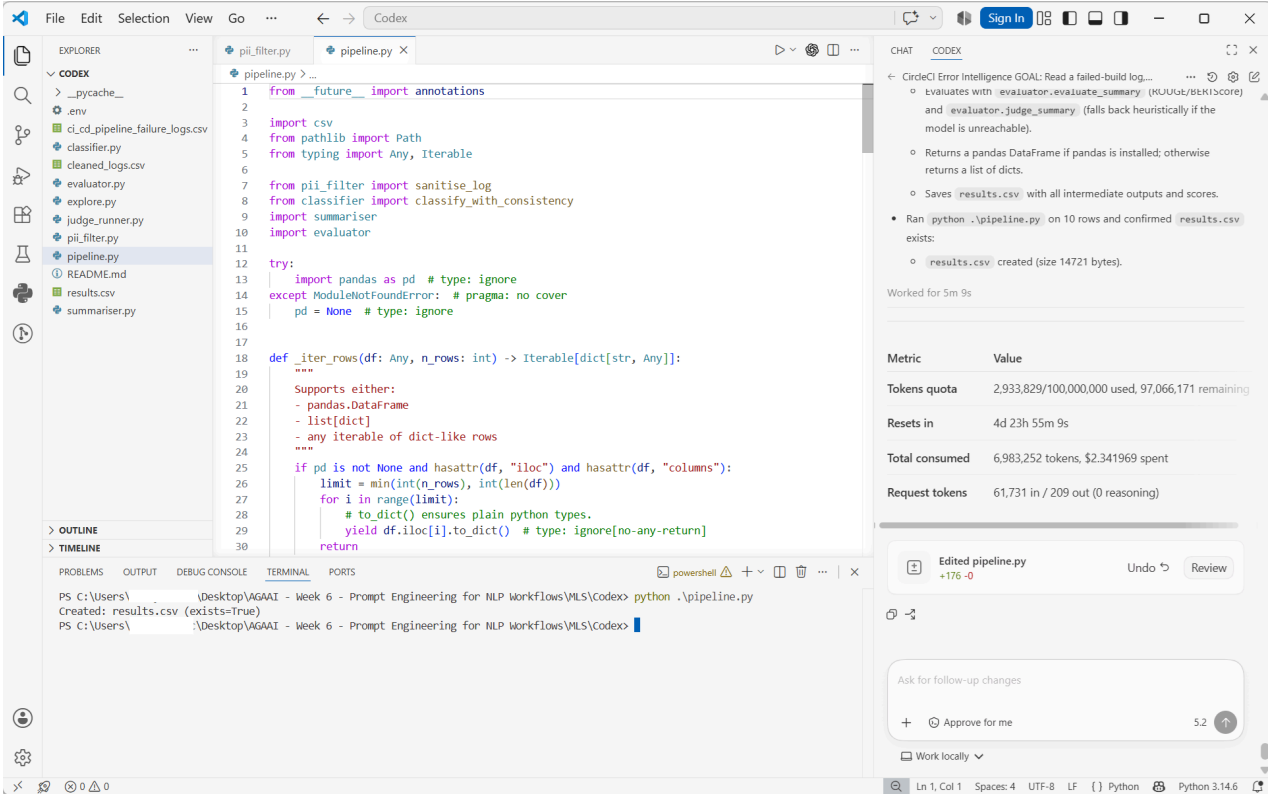
IN PLAIN ENGLISH

So far each stage is a separate script. Now you connect them: sanitise, classify with self-consistency, summarise with the engineered prompt, and evaluate with the judge, all in one run that writes a results file.

Then the real lesson: load the results, find the lowest-actionability summaries, ask why they are weak, change the prompt to fix that specific weakness, and re-measure. Generate, review, refine, applied to the prompt itself.

PROMPT 7A: WIRE THE FULL PIPELINE

Add `full_pipeline(df, n_rows=10)` to `pipeline.py` that: takes `n_rows`; sanitises `error_message`; classifies with `classify_with_consistency`; generates a summary with the persona + output spec prompt; evaluates with both `evaluate_summary` and `judge_summary`; returns a results dataframe with all intermediate outputs and scores; saves `results.csv`. Run on 10 rows and confirm `results.csv` was created.



The screenshot shows a VS Code editor with the `pipeline.py` file open. The file contains the following code:

```
1 from __future__ import annotations
2
3 import csv
4 from pathlib import Path
5 from typing import Any, Iterable
6
7 from pii_filter import sanitise_log
8 from classifier import classify_with_consistency
9 import summariser
10 import evaluator
11
12 try:
13     import pandas as pd # type: ignore
14 except ModuleNotFoundError: # pragma: no cover
15     pd = None # type: ignore
16
17
18 def iter_rows(df: Any, n_rows: int) -> Iterable[dict[str, Any]]:
19     """
20     Supports either:
21     - pandas.DataFrame
22     - list[dict]
23     - any iterable of dict-like rows
24     """
25     if pd is not None and hasattr(df, "iloc") and hasattr(df, "columns"):
26         limit = min(int(n_rows), int(len(df)))
27         for i in range(limit):
28             # to dict() ensures plain python types.
29             yield df.iloc[i].to_dict() # type: ignore[no-any-return]
30     return
```

The chat window on the right shows the following summary of the pipeline's execution:

- CircleCI Error Intelligence GOAL: Read a failed-build log...
- evaluates with `evaluator.evaluate_summary` (KQ/UE/BE/IS/Score) and `evaluator.judge_summary` (falls back heuristically if the model is unreachable).
- Returns a pandas DataFrame if pandas is installed; otherwise returns a list of dicts.
- Saves `results.csv` with all intermediate outputs and scores.
- Ran `python .\pipeline.py` on 10 rows and confirmed `results.csv` exists:
 - `results.csv` created (size 14721 bytes).

Worked for 5m 9s

Metric	Value
Tokens quota	2,933,829/100,000,000 used, 97,066,171 remaining
Resets in	4d 23h 55m 9s
Total consumed	6,983,252 tokens, \$2.341969 spent
Request tokens	61,731 in / 209 out (0 reasoning)

Edited `pipeline.py` +176 -0 Undo Review

Ask for follow-up changes

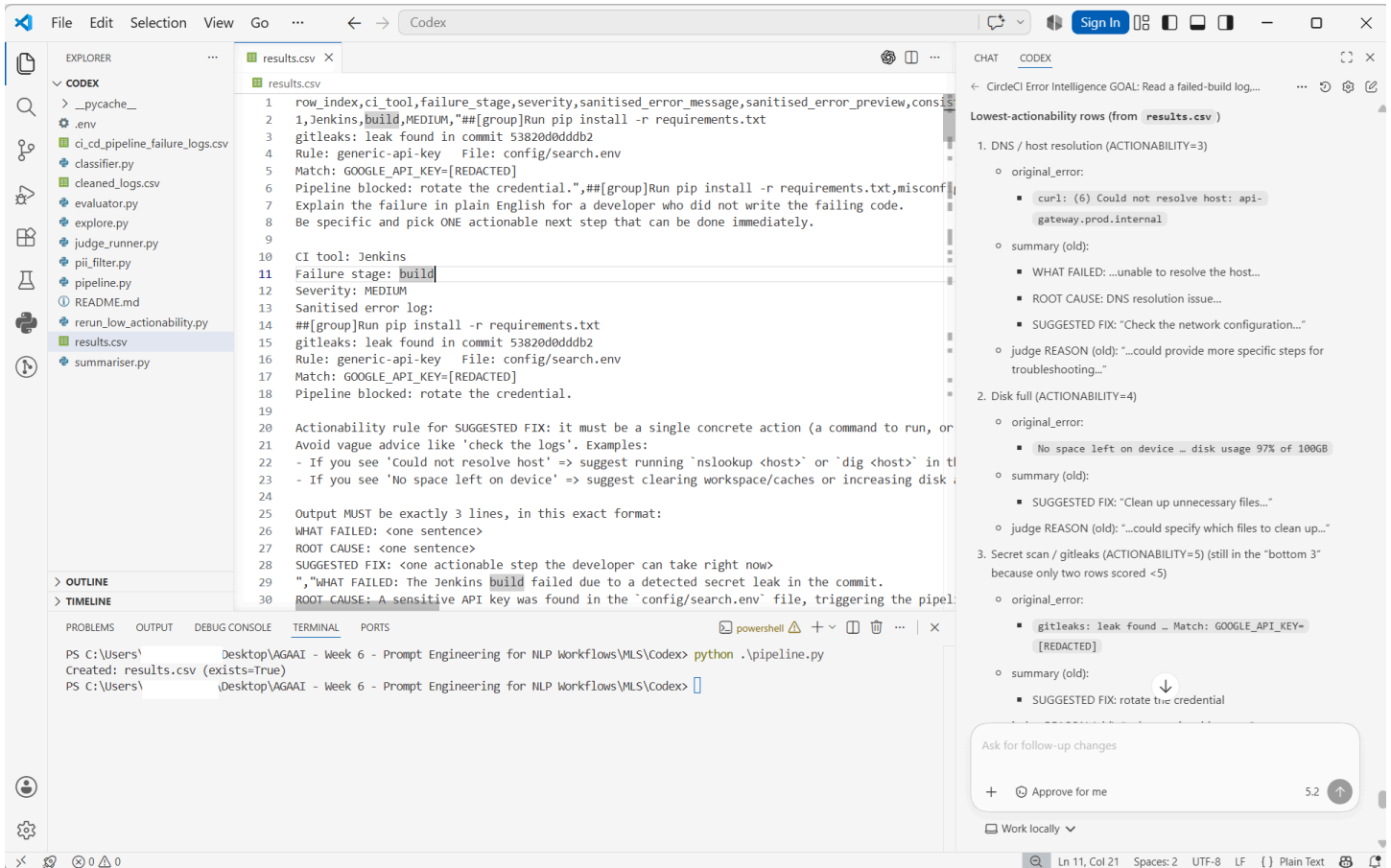
+ Approve for me 5.2

Work locally

Ln 1, Col 1 Spaces: 4 UTF-8 LF {} Python Python 3.14.6

PROMPT 7B: IMPROVE THE PROMPT FROM THE EVALUATION

Load results.csv and find the 3 rows with the lowest ACTIONABILITY score. Print original_error, summary, and the judge's REASON, then: (1) what do the low-scoring messages have in common; (2) what is missing from the PromptTemplate that caused weak fixes; (3) propose one specific change - a new instruction, example, or output field - to improve actionability for these cases. Implement it in summariser.py, re-run those 3 rows, and print old vs new summary and new judge scores. Did actionability improve?



The screenshot shows a VS Code editor with the following components:

- EXPLORER:** A file tree on the left showing a project structure with files like `_pycache_`, `env`, `ci_cd_pipeline_failure_logs.csv`, `classifier.py`, `cleaned_logs.csv`, `evaluator.py`, `explore.py`, `judge_runner.py`, `pii_filter.py`, `pipeline.py`, `README.md`, `rerun_low_actionability.py`, `results.csv`, and `summariser.py`.
- results.csv:** A CSV file with columns: `row_index`, `ci_tool`, `failure_stage`, `severity`, `sanitised_error_message`, `sanitised_error_preview`, and `actionability`. The data includes rows for Jenkins build failures, git leaks, and disk full errors.
- CHAT:** A chat window on the right titled "CircleCI Error Intelligence GOAL: Read a failed-build log...". It displays the "Lowest-actionability rows (from results.csv)" and provides analysis and suggestions for three specific rows:
 - DNS / host resolution (ACTIONABILITY=3):** The original error was a curl command failing to resolve a host. The summary (old) was "unable to resolve the host...". The judge's reason (old) was "could provide more specific steps for troubleshooting...".
 - Disk full (ACTIONABILITY=4):** The original error was "No space left on device". The summary (old) was "Clean up unnecessary files...". The judge's reason (old) was "could specify which files to clean up...".
 - Secret scan / git leaks (ACTIONABILITY=5):** The original error was a git leak found. The summary (old) was "rotate the credential".
- TERMINAL:** A terminal window at the bottom showing the command `python .\pipeline.py` being executed.

CHECK YOUR OUTPUT BEFORE YOU MOVE ON

Confirm full_pipeline writes results.csv with the intermediate columns and the judge scores: a 10-row table with per-row judge_accuracy and judge_actionability.

On the improvement: expect the weakest cases to be opaque, heavily redacted logs where the model was guessing. A strong proposed fix is a new required output field, EVIDENCE GAP (the single most important missing log or command output needed next), plus an instruction not to guess from insufficient evidence. Re-running the three weakest rows should lift average actionability from 1.33 to 2.0.

Read the proposed prompt change critically before accepting it. The point is that you can justify why the change should help, not just that the number went up.

WHAT YOU SHOULD SEE

- ✓ results.csv created; a 10-row table with predicted_category, judge_accuracy, judge_actionability
- ✓ The 3 lowest-actionability rows printed with the judge's reason
- ✓ A specific, justified prompt change (e.g. an EVIDENCE GAP field plus an anti-guessing instruction)
- ✓ Old vs new comparison on those rows; average actionability rising from 1.33 to 2.0

WATCH OUT

An AI will happily report "actionability improved" from a tiny sample. Three rows is a demonstration, not proof. Name that limit: the loop is right, but the evidence is small.

When improving a prompt, the AI may rewrite far more than needed. Insist on one targeted change so you can attribute the improvement to it, otherwise you cannot tell what worked.

KEY INSIGHT

This is the whole module in one move: measure the summaries, find the worst ones, ask why, change the prompt to fix that exact weakness, then measure again. Prompt engineering is not writing one clever prompt. It is this loop.

REFLECT

20. The fix was a new EVIDENCE GAP field that admits what is missing. Why is an honest "I cannot tell without X" often more actionable than a confident guess?
21. Actionability rose from 1.33 to 2.0 on three rows. Why is that not yet proof, and how would you prove it properly?
22. Which single stage of this pipeline would you still want a human to write by hand, and why?

8. Putting it all together: the full NLP workflow

IN PLAIN ENGLISH

Zoom out and look at the complete picture. You started with a business problem (about 20 minutes per failed build, 34% of tickets diagnostic) and a raw failure-log file. You ended with a working NLP pipeline that sanitises, classifies, summarises, and self-evaluates, and a demonstrated way to improve it. And you built every line by prompting Codex, reviewing its output, and refining.

Two lifecycles ran in parallel: the NLP workflow (data → sanitise → classify → summarise → advanced prompting → evaluate → improve) and the AI-assisted coding loop (generate → review → refine) repeated at every step. The second is the transferable skill. It works for any software, not just NLP.

#	Stage	What it delivered (and the lesson)
1	Data exploration	Profiled the file and caught the 25-vs-28 column mismatch. Lesson: trust the data over the spec.
2	PII filtering	Built the sanitisation guardrail before any model call. Lesson: constraints protect the business; redaction quality flows downstream.
3	Classification	Zero-shot vs few-shot on the fixed label set. Lesson: how you ask changes the answer, and prompts sit downstream of data quality.
4	Summarisation	Basic template, then persona + output spec. Lesson: a role and a required shape turn vague text into an actionable fix.
5	Advanced patterns	Chain-of-Thought + self-consistency. Lesson: reasoning and voting buy reliability; high agreement on no signal is not correctness.
6	Evaluation	ROUGE/BERTScore failed honestly; pivoted to LLM-as-a-Judge. Lesson: the right metric is a skill. Engineered prompts scored 2.0 vs 1.2.
7	Pipeline + feedback loop	Wired it together; improved the weakest prompt via an EVIDENCE GAP field (1.33 to 2.0). Lesson: prompt engineering is a measure-and-refine loop.

KEY INSIGHT

The meta-skill: the pipeline you built is disposable. The way of working is the asset. The AI generated everything; your value was knowing what to ask for, spotting that the metric was wrong, insisting on the output structure, and improving the prompt from the evidence. That judgement is what you are paid for in an AI world.

9. Deeper insights: questions you might have

Question	Answer
Why not just keyword-match the logs instead of using an LLM?	Keyword rules are brittle and cannot explain or suggest a fix. The same error string can mean different things in context, and new error formats appear constantly. An LLM generalises and produces a plain-English explanation a rule cannot.
Why did the classifier return unknown so often?	On this synthetic dataset the PII filter redacted almost all of the error text, leaving little to classify. It is a data-quality effect flowing downstream, not a flaw in the prompt. With logs whose error text survives sanitisation, few-shot separates the labels far better.
Why were ROUGE and BERTScore useless here?	ROUGE needs a human reference summary to overlap against; you only had the raw log, so overlap was zero. BERTScore needs a heavy embedding runtime that was not available, so it returned nan. Both told you the metric did not fit the task, which is why you moved to LLM-as-a-Judge.
Is LLM-as-a-Judge reliable enough to trust?	It is a useful proxy when no reference exists, but it has biases and needs a strict rubric and format. In production you calibrate it against human ratings on a sample and use it to triage, not as the final word.
Why does few-shot beat zero-shot at all?	The examples demonstrate the exact label schema and the mapping you want, which reduces the model's interpretation drift. You are teaching by example inside the prompt itself.
What did persona + output spec actually change?	Persona set the seniority and tone; the output spec forced a parseable WHAT FAILED / ROOT CAUSE / SUGGESTED FIX shape. Together they lifted judged actionability from 1.2 to 2.0 versus the basic template.
Why run the model five times for self-consistency?	A single reasoning run can land on an unlucky answer for an ambiguous log. Voting across runs stabilises the label. The cost is five calls per row, which is why you run it once rather than repeatedly.
What was the EVIDENCE GAP fix in Stage 7?	A new required output field naming the single most important missing log or command output, plus an instruction not to guess from thin evidence. It stopped the model inventing causes for opaque logs and raised actionability from 1.33 to 2.0 on the weakest rows.

9.1 Questions about the AI-assisted coding itself

Question	Answer
Can the AI just build the whole thing without me?	It can produce something that runs, but in this build it defaulted to the wrong evaluation metric and reported zeros and nan without flagging the mismatch, and it guessed causes for opaque logs until the EVIDENCE GAP field was added. The AI optimises for what you ask; you own the spec, the review, and the judgement.
How do I know the AI's prompt is actually good?	Measure it. The whole module is the answer: define a task-appropriate metric (actionability), score the output, find the weak cases, change the prompt, re-measure. A prompt is good when the evidence says so, not when it reads well.

Question	Answer
What if Codex gives different code each time I ask?	It often will, and that is expected. What stays constant is your spec and your checks: sanitise first, fixed label set, fixed output shape, the right metric. You are not memorising one correct output; you are enforcing the constraints any correct output must satisfy.
Is learning the fundamentals obsolete now?	The opposite. You can only review what you understand. Spotting that ROUGE was the wrong metric, knowing why few-shot helps, recognising that high vote-confidence on no signal is meaningless, all of that comes from understanding NLP. The AI raises the value of fundamentals.

10. Troubleshooting: if something goes wrong when you run it

Things that can go wrong as you build, and how to recover.

Symptom	Cause	Fix
Every classification comes back unknown	API key or model not set, or text fully redacted in Stage 2	Check the model call is succeeding first (an unset key silently falls back to unknown). If calls work, it is the data: your redaction removed the signal, a data lesson, not a prompt failure.
Gateway rejects the model ("model not allowed")	The default model name is not permitted on your gateway	Set OPENAI_MODEL to an allowed model. Do this during setup, before you start building.
ROUGE all zeros / BERTScore nan	Wrong metric for the task; embedding runtime unavailable	This is the Stage 6 lesson, not a bug. The metric does not fit, so pivot to LLM-as-a-Judge.
Summaries break the WHAT FAILED format	Output spec not strong enough	Give one refinement prompt restating the exact three-part structure. Models drift without an explicit shape.
Codex generates wrong or off-task code	Prompt under-specified, or normal AI variance	A teaching moment, not a failure. Name what is wrong and give one refinement prompt. If the second attempt also misses, step back and re-scope the prompt.
ModuleNotFoundError (langchain, bert-score, etc.)	Package not installed in the active environment	Run pip install in the terminal. Prevention: pre-install everything during setup (Section 1).
A script references an API key in code	Key accidentally hardcoded	Stop and remove it. Keys live in environment variables only, never in a script, prompt, or slide.
Your numbers differ slightly from this guide	Fresh generation and model variance	Trust your own numbers. Minor variation is normal; the magnitude and the conclusion are unchanged.

11. Known limitations: what to keep in mind

Part of being a credible practitioner is knowing and communicating the limits of your own work. Flag limitations proactively; it builds trust.

Limitation	Plain-English explanation	What would fix it
Synthetic dataset	The logs are real-world-inspired but largely random token text, so after PII redaction there is little real error language to classify or summarise.	Use real (or realistically formatted) CI/CD logs whose error text survives sanitisation.
Over-aggressive redaction	The PII filter redacts almost everything on this data, which starves the downstream NLP stages.	Tune the regex to target genuine secrets while preserving the surrounding error text.
Tiny evaluation sample	Judge scores and the Stage 7 improvement were measured on a handful of rows, a demonstration, not statistical proof.	Score a few hundred rows and compare techniques with confidence intervals.
LLM-as-a-Judge bias	Using a model to grade a model can inherit the model's blind spots.	Calibrate judge scores against human ratings on a sample; use multiple judges.
No production reference summaries	You have no human-written gold summaries, which is why ROUGE/BERTScore did not apply.	Curate a small set of expert-written explanations as references for a proper overlap and semantic evaluation.

12. Key numbers and takeaways

A quick reference for the key numbers and decisions in this module.

Key fact / number	What it means
45,000 rows x 25 columns	The raw dataset. Your prompt claimed 28 columns; the data has 25. Trust the data.
About 9,007 CircleCI rows	Your focus platform, though the data spans five CI/CD tools.
10 failure types, 3 failure stages	The natural label structure in the data; your classifier uses a fixed 8-label taxonomy plus unknown.
Sanitise before any model call	PII (keys, tokens, paths) must be redacted first. On this data redaction is near-total, a quirk that flows downstream.
Zero-shot vs few-shot	Same model, same input; examples in the prompt anchor the label schema and cut drift.
Persona + output spec	A role plus a fixed WHAT FAILED / ROOT CAUSE / SUGGESTED FIX shape turns vague text into an actionable fix.
ROUGE 0.0, BERTScore nan	Not a bug: the wrong metric (no reference summary) and an unavailable runtime. The cue to pivot to LLM-as-a-Judge.
Actionability: basic 1.2, persona 2.0, CoT 2.0	The payoff of prompt engineering, measured on the metric that matters.
Self-consistency = 5 calls per row	Reasoning runs plus a majority vote; reliable but slow, so run once rather than repeatedly.
EVIDENCE GAP fix: 1.33 to 2.0	Adding a field that names the missing evidence (and banning guessing) lifted actionability on the weakest rows.
Generate → Review → Refine	The loop behind every stage. Generate = prompt; Review = read + check against Section 5.4; Refine = one targeted change. You own every prompt you ship.
Never show a real API key	Keys live in environment variables only; use a placeholder if projecting.

End of Handbook